

Jogo da Forca Distribuído

Documentação de construção e guia de estudo

Python • sockets TCP • salas • turnos • replicação • VMs • Docker

Versão do projeto: 0.2.0 | Código consultado em 27/09/2026

Este material explica a implementação existente, relaciona cada requisito ao código e mostra como demonstrar o sistema. Foi escrito para estudantes que estão aprendendo redes e sistemas distribuídos. Os exemplos acompanham o projeto `forca-sistemas-distribuidos`, revisão Git `1ecf320`.

O foco principal é socket: como uma intenção digitada em um terminal vira bytes, chega ao servidor, modifica uma partida e aparece nos terminais dos dois jogadores. A segunda parte central é a cópia desse estado para outro computador, antes de confirmar uma jogada.

Sumário

1. O que foi construído e como estudar	4
2. Os seis requisitos e onde estão no código	5
3. Arquitetura e divisão dos arquivos	6
4. Socket explicado desde o início	7
5. O ciclo de vida dos sockets no código	9
6. Como as mensagens são separadas e interpretadas	11
7. Uma jogada completa, do teclado aos dois terminais	12
8. Vários jogadores, salas e limite de duas pessoas	14
9. O servidor como semáforo: três mecanismos	15
10. Estados, erros e os dois bonecos	17
11. Replicação: como a cópia chega à outra máquina	19
12. O que acontece quando uma máquina cai	20
13. Sessões, identidade e comandos pendentes	22
14. Como executar com Python, VMs e Docker	23
15. Testes e roteiro para apresentar ao professor	25
16. Dúvidas comuns e diagnóstico	26
17. Glossário e mapa de leitura do código	28
18. Referências e rastreabilidade	29
Apêndice A. Código integral da versão documentada	30
forca/wire.py	30
servidor.py	31

cliente.py	34
forca/game.py	37
forca/lobby.py	39
forca/ui.py	41
forca/__init__.py	42
forca/words.txt	43
Dockerfile	43
compose.yaml	43
.env.example	43
.dockerignore	44
pyproject.toml	44
scripts/forca.ps1	45
scripts/gerar_especificacao_pdf.py	46
tests/test_game.py	48
tests/test_lobby.py	50
tests/test_cliente.py	53
tests/test_servidor.py	54
.gitignore	57

1. O que foi construído e como estudar

O projeto é um jogo de força para vários jogadores, organizados automaticamente em salas com no máximo duas pessoas. Cada sala tem uma palavra compartilhada, uma vez de jogar e dois contadores de erros. Cada participante possui seu próprio boneco, e os dois bonecos aparecem nos dois clientes.

Um cliente é o programa que o jogador abre no terminal. O servidor é o programa que recebe os pedidos e decide o que pode acontecer. Os terminais de Ana e Bruno não conversam diretamente entre si: ambos conversam com o servidor. A palavra secreta e as regras ficam no servidor.

Existem duas instâncias do programa servidor: o primário atende os jogadores e o reserva recebe cópias do estado. Instância significa uma execução do mesmo programa. Quando o primário cai, o reserva sincronizado assume o atendimento, e os clientes tentam seu endereço. O jogo continua depois de uma interrupção de alguns segundos, preservando o estado que foi replicado.

O projeto atual utiliza Python e sua biblioteca padrão para executar o jogo. Não utiliza Supabase, banco central, serviço de nuvem nem processo separado para cada sala. A configuração de entrega usa Docker em duas VMs, instaladas em dois computadores físicos distintos.

1.1 O que este documento comprova

A explicação foi construída pela leitura do código atual. A ordem dos capítulos é uma ordem didática de construção, e não uma alegação sobre a ordem histórica em que cada integrante escreveu os arquivos. Os trechos destacados são extraídos automaticamente dos arquivos reais. No apêndice há uma cópia integral do código e das configurações selecionadas, com números de linha.

Em 27/09/2026, os 50 testes automatizados do projeto foram executados neste ambiente Windows, com resultado OK em 8,382 segundos. Isso comprova os cenários cobertos pelos testes locais; não comprova desligamento físico de computador, funcionamento de uma rede específica ou execução dos containers nas duas VMs. Essa etapa continua sendo uma validação prática do grupo.

1.2 Caminho sugerido para aprender

1. Entenda os componentes e os seis requisitos nos capítulos 2 e 3.
2. Estude sockets, mensagens e o caminho da jogada nos capítulos 4 a 7.
3. Veja salas, turnos e bonecos nos capítulos 8 a 10.
4. Estude replicação, falhas e sessões nos capítulos 11 a 13.
5. Execute os roteiros dos capítulos 14 e 15 e consulte o código integral no apêndice.

Quando aparecer um nome como `Server.request`, leia como “o método `request` da classe `Server`”. Uma classe reúne dados e funções relacionados; um método é uma função definida dentro dela. Um dicionário Python associa chaves a valores, por exemplo `{"errors": 2}`.

2. Os seis requisitos e onde estão no código

Requisito do trabalho	Implementação atual	Como demonstrar
Utilizar socket	TCP em cliente.py e servidor.py; mensagens em forca/wire.py	Dois clientes em uma máquina acessam o IP de outra
Servidor resiliente	Cópia síncrona para o reserva; Server.follow promove o reserva	Desligar o computador do primário depois da sincronização
Múltiplas requisições e espera	Threads atendem conexões; lobby.enter distribui jogadores	Abrir cinco clientes e observar três salas
Máximo de dois jogadores	Busca apenas salas com uma vaga; Room.start exige dois	Ana/Bruno ficam juntos; Caio/Dani ficam em outra sala
Controle como semáforo	Lock serializa processamento; turn autoriza o jogador	Enviar letra fora da vez e verificar a rejeição
Estados e dois bonecos	Resposta inclui erros de ambos; ui.render desenha ambos	Ana erra: seu boneco muda nos dois terminais

“Atendido no código” e “validado no ambiente de apresentação” são coisas diferentes. As regras têm testes automatizados. A recuperação de falha é testada com processos locais. A demonstração exigida pelo professor precisa usar computadores físicos diferentes, para que desligar um não desligue as duas VMs.

2.1 Precisão sobre o requisito de resiliência

Esta versão tolera a queda do primário, desde que o reserva já esteja sincronizado e a rede dos clientes continue funcionando. Se o reserva cair primeiro, o primário pausa para não confirmar novas alterações sem uma segunda cópia. Após assumir, o reserva trabalha sozinho. Portanto, a tolerância a falhas é limitada a esse cenário e não cobre indiscriminadamente a queda de qualquer máquina em qualquer momento.

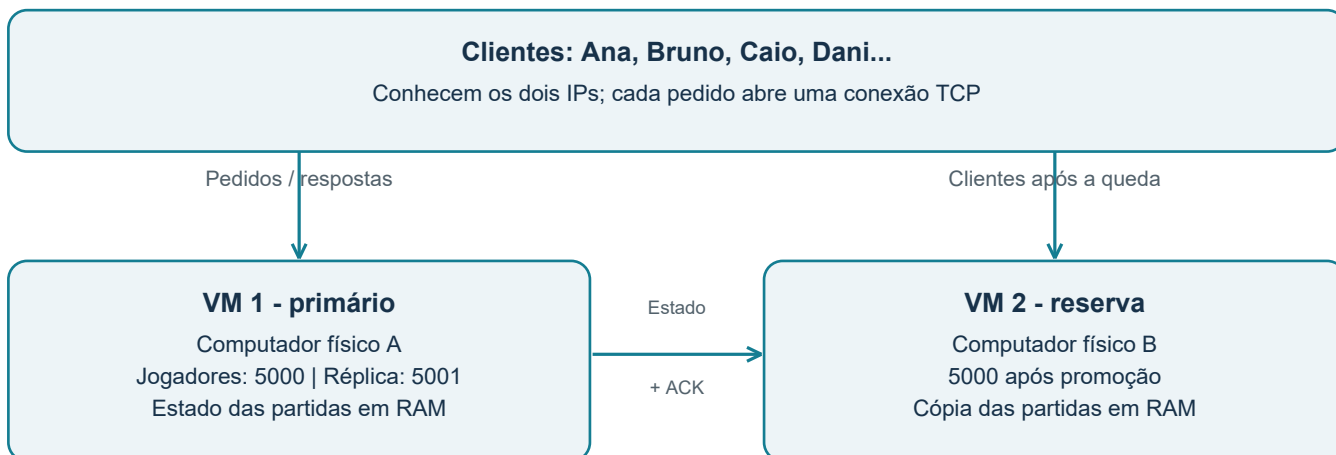
Se o professor exigir que qualquer um dos dois servidores possa cair sem interromper o jogo, a versão atual ainda não atende integralmente a essa interpretação. É essencial apresentar a garantia real, em vez de chamar qualquer recuperação de “100% disponível”. O capítulo 12 detalha os casos.

2.2 Três limites que não devem ser confundidos

Dois é a quantidade máxima de jogadores por sala. Sessenta e quatro é o limite de conexões atendidas simultaneamente pelo servidor. Um é o número de pedidos que podem estar dentro da região protegida por Lock ao mesmo tempo. Esses números tratam de problemas diferentes.

O projeto pode manter várias salas. Uma sala não ganha uma porta exclusiva, uma VM própria ou um processo próprio. Todas usam o mesmo servidor e a mesma porta de jogadores. O servidor descobre a sala pelo jogador identificado no pedido.

3. Arquitetura e divisão dos arquivos



Canal entre servidores: TCP persistente, iniciado pelo reserva até a porta 5001.

A figura mostra o funcionamento normal. O reserva inicia a conexão de replicação com o primário e a mantém aberta. Os clientes conhecem os dois IPs, mas o reserva só abre o atendimento de jogadores depois da promoção. As setas representam comunicação TCP; não representam memória compartilhada.

Arquivo	Responsabilidade
servidor.py	Aceitar sockets, coordenar threads, proteger o estado, replicar e assumir o atendimento
cliente.py	Ler teclado, enviar pedidos, mostrar respostas, salvar sessão e reconectar
forca/wire.py	Converter dicionários em mensagens JSON e ler uma mensagem por linha
forca/game.py	Representar jogadores e salas; validar letras, turnos, erros e vitória
forca/lobby.py	Distribuir jogadores em salas, reservar nomes e reconhecer reenvios
forca/ui.py	Transformar o estado público em texto e desenhar as forcas
forca/words.txt	Lista de palavras sorteadas pelo servidor
Dockerfile e compose.yaml	Preparar e executar o servidor em container
tests/	Testar regras, cliente, protocolo e troca de servidor

3.1 Construção em camadas

Primeiro podemos entender o jogo sem rede: Room, em game.py, sabe iniciar uma partida e processar uma tentativa. Depois vem o lobby, que escolhe uma sala e associa os jogadores. Nenhum desses dois módulos abre sockets. Isso permite testar regras com chamadas de funções, sem depender de uma VM.

Em seguida, `wire.py` define como os dados serão transportados. `servidor.py` liga a comunicação às regras: recebe uma mensagem, aplica a operação e devolve uma resposta. `cliente.py` cuida da interação com a pessoa. Por fim, a replicação acrescenta uma segunda cópia do estado e o Docker organiza a execução.

Separar responsabilidades ajuda a aprender e a corrigir problemas. Um desenho errado pode estar em `ui.py`; uma letra aceita fora da vez pode estar em `game.py`; uma mensagem que não chega pode envolver `cliente.py`, `servidor.py`, `wire.py` ou a rede.

3.2 O que torna o sistema distribuído

Cada processo tem sua própria memória. O primário não acessa diretamente a RAM do reserva. Para que o reserva conheça uma partida, o primário precisa transformar seus dados em uma mensagem e enviá-la pela rede. As máquinas podem atrasar, perder comunicação ou desligar em momentos diferentes.

O mesmo programa pode executar em mais de um computador, mas isso sozinho não garante continuidade. A continuidade depende de cópias do estado, confirmação de recebimento, detecção de falha e reconexão dos clientes. Esses mecanismos são parte explícita do código deste projeto.

4. Socket explicado desde o início

Um socket é um recurso do sistema operacional que um programa usa para se comunicar. Pense nele como uma ponta de comunicação. Em uma conversa TCP há uma ponta no cliente e outra no servidor. Python oferece a biblioteca `socket` para pedir ao sistema operacional que crie, conecte, escute e feche essas pontas.

Um socket não é uma sala de jogo, um terminal nem o arquivo JSON da sessão. O terminal recebe o texto digitado; `cliente.py` decide o comando; o socket transporta os bytes desse comando. O servidor interpreta os bytes e encontra o jogador e a sala correspondentes.

4.1 IP, porta e conexão

O IP identifica um endereço de rede. A porta identifica o ponto de atendimento dentro daquele endereço. No exemplo `192.168.1.10:5000`, o cliente procura a porta TCP 5000 da máquina com IP `192.168.1.10`.

Endereço ou porta	Significado neste projeto
127.0.0.1	A própria máquina ou ambiente de rede em que o processo está rodando
0.0.0.0 no servidor	Escutar nas interfaces IPv4 locais; não é o endereço a informar ao jogador
IP da VM	Endereço usado por outra máquina para chegar ao servidor na VM
5000	Atendimento de jogadores, por padrão
5001	Canal de replicação do primário, por padrão
5002	Porta usada para o reserva no teste com ambos no mesmo computador

O cliente também tem uma porta local, normalmente escolhida automaticamente pelo sistema operacional. Não é necessário reservar uma porta manual para Ana e outra para Bruno. Várias conexões podem chegar à porta 5000: elas são diferenciadas pelos endereços e portas das duas pontas.

Dentro de uma VM, 127.0.0.1 aponta para a VM. Dentro de um container, aponta para o ambiente de rede do container. Por isso, o reserva em outra VM deve receber o IP real do primário em `--primario` ou `PRIMARY_HOST`.

4.2 Por que TCP

TCP oferece um fluxo de bytes ordenado durante uma conexão. O sistema operacional cuida de detalhes como retransmitir segmentos perdidos. O projeto usa esse transporte para mensagens pequenas de comando e estado.

Isso não significa que TCP salva uma partida ou garante que uma jogada foi aplicada. Se o computador desligar, a conexão pode falhar. Mesmo um envio sem erro não prova que o programa remoto terminou de processar a mensagem. O protocolo do jogo precisa de respostas e, na replicação, de um ACK próprio da aplicação.

O ACK enviado em JSON pelo reserva é uma confirmação do programa: “ guardei o estado desta revisão”. Ele é diferente das confirmações internas do protocolo TCP. Essa diferença explica por que o projeto espera uma mensagem explícita antes de confirmar a jogada.

4.3 TCP transporta bytes, não dicionários

Um dicionário Python não atravessa a rede como um objeto da memória. O projeto usa esta sequência:

```
Dicionário Python -> texto JSON -> bytes UTF-8 -> socket TCP  
socket TCP -> bytes recebidos -> leitura do JSON -> dicionário Python
```

JSON é um formato de texto para representar objetos, listas, números e outros valores. UTF-8 é uma forma de representar caracteres como bytes, incluindo letras acentuadas. Serializar é converter dados para uma representação que possa ser enviada; desserializar é reconstruir os dados a partir dela.

As referências oficiais de socket ajudam a aprofundar esses conceitos [R1, R2]. Os próximos capítulos aplicam cada um diretamente às funções do projeto.

5. O ciclo de vida dos sockets no código

5.1 O servidor cria um ponto de escuta

Trecho real: *servidor.py* • *Server.listen* • linha 98

```
98 def listen(self, port):
99     listener = socket.socket()
100     listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
101     listener.bind((self.host, port))
102     listener.listen(64)
103     return listener
```

`socket.socket()` cria, com os padrões usados aqui, um socket IPv4 TCP. A forma explícita seria `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`. IPv4 é a família dos endereços como 192.168.1.10; `SOCK_STREAM` indica o transporte por fluxo usado com TCP.

`setsockopt` configura uma opção do socket. `SO_REUSEADDR` ajuda no reaproveitamento do endereço em situações de reinício, conforme as regras do sistema operacional. Não significa que se pode executar dois primários arbitrariamente na mesma porta.

`bind` associa o socket ao endereço local e à porta. `listen` o coloca em modo de escuta. O 64 em `listen(64)` é uma indicação de tamanho da fila de conexões aguardando aceitação pelo programa, sujeita ao sistema operacional. Essa fila não é a sala de espera do jogo.

5.2 accept cria a conexão de atendimento

Trecho real: *servidor.py* • *Server.serve* • linha 105

```
105 def serve(self):
106     with self.listen(self.port) as listener:
107         LOG.info("Atendendo jogadores em %s:%s", self.host, self.port)
108         while True:
109             connection, _ = listener.accept()
110             if self.slots.acquire(blocking=False):
111                 threading.Thread(target=self.client, args=(connection,), daemon=True).start()
112             else:
113                 connection.close()
```

`accept` espera uma conexão e devolve um novo socket conectado. `listener` continua disponível para novas conexões; `connection` serve à conversa que acabou de chegar. O segundo valor devolvido é o endereço remoto, ignorado neste trecho pelo nome `_`.

Para cada conexão admitida, uma thread executa `self.client`. Uma thread é uma linha de execução dentro do mesmo processo. Enquanto uma espera bytes de um jogador, o laço de atendimento pode aceitar outra conexão. As threads compartilham o estado do servidor, por isso precisam da proteção explicada no capítulo 9.

O `BoundedSemaphore` limita a 64 os atendimentos simultâneos. `acquire(blocking=False)` tenta ocupar uma vaga sem esperar. Se não houver vaga, a conexão é fechada. No finally de `Server.client`, `release` devolve a vaga, inclusive em caso de erro. Não são 64 jogadores para sempre: são 64 conexões em atendimento naquele instante.

5.3 O cliente conecta, envia, recebe e fecha

Trecho real: cliente.py • exchange • linha 23

```
23 def exchange(address, command):
24     host, port = address.rsplit(":", 1)
25     with socket.create_connection((host, int(port)), timeout=2) as connection:
26         connection.settimeout(5)
27         with connection.makefile("rwb") as stream:
28             send(stream, command)
29             return receive(stream)
```

`address` é um texto como `192.168.1.10:5000`. `rsplit` separa o IP da porta, e `int` converte a porta para número. `create_connection` abre a conexão, com limite de dois segundos para a tentativa de conexão. `settimeout(5)` define o limite usado nas operações seguintes do socket.

`makefile("rwb")` cria uma interface parecida com um arquivo sobre o socket: `r` permite ler, `w` permite escrever, `b` significa bytes. Não cria um arquivo no disco. A variável `stream` representa esse acesso à conexão, usado por `send` e `receive` de `wire.py`.

Os blocos `with` fecham os recursos ao terminar, também quando há exceção. Cada consulta ou comando do jogador usa uma conexão nova. O cliente não mantém uma única conexão por toda a partida.

5.4 O servidor trata a mensagem

`Server.client` configura quatro segundos de `timeout`, cria seu `stream`, lê um objeto com `receive`, chama `Server.request` e envia a resposta com `send`. Erros de estrutura recebem uma resposta controlada. Erros inesperados são registrados no log e geram uma mensagem genérica ao cliente. Se a conexão já estiver rompida, enviar a resposta também pode falhar.

`Timeout` é um limite de espera de uma operação; não é um cronômetro que garante o tempo total de recuperação. Somam-se tentativas em outros endereços, processamento, pausas e condições da rede. O cliente trata falhas de conexão tentando o próximo servidor.

6. Como as mensagens são separadas e interpretadas

6.1 O problema das mensagens partidas

TCP não preserva as fronteiras das mensagens do programa. Mesmo que o emissor envie um JSON de uma vez, o receptor pode obter seus bytes em partes. Duas mensagens seguidas também podem estar disponíveis juntas. Por isso, uma leitura simples de um bloco de bytes não equivale necessariamente a uma mensagem completa.

Mensagem enviada: {"type":"PING"}\n
 Possível chegada em partes: {"ty + pe":"PI + NG"}\n
 Outra possibilidade: duas linhas chegam juntas no buffer.

O projeto escolheu uma regra: cada objeto JSON termina com uma quebra de linha, representada por \n. Essa regra de separação é chamada enquadramento. Os textos são serializados sem indentação; quebras de linha dentro de uma string JSON são escapadas pelo serializador e não viram delimitadores extras da mensagem.

6.2 O envio, linha por linha

Trecho real: *forca/wire.py* • *send* • linha 7

```
7 def send(stream, message):
8     data = json.dumps(message, ensure_ascii=False).encode("utf-8") + b"\n"
9     if len(data) > MAX_BYTES:
10         raise ValueError("Estado grande demais para o limite desta demonstração.")
11     stream.write(data)
12     stream.flush()
```

`json.dumps` transforma o dicionário em texto JSON. `ensure_ascii=False` permite representar os caracteres diretamente no texto. `encode("utf-8")` transforma o texto em bytes. O `b"\n"` acrescenta o delimitador em bytes ao final.

Antes de escrever, o código limita o tamanho a 2.000.000 bytes, incluindo o delimitador. `stream.write` entrega os bytes ao stream. `flush` descarrega o buffer de escrita, isto é, manda adiante o que poderia estar esperando na memória local do stream. Sem essa etapa, o outro lado poderia continuar esperando uma mensagem que ainda não saiu do buffer.

O nome `send` aqui é uma função criada no projeto; não é uma chamada direta ao método `socket.send`. A comunicação continua sendo por `socket`: `write` e `flush` operam sobre o stream criado a partir dele.

6.3 A leitura, linha por linha

Trecho real: *forca/wire.py* • *receive* • linha 15

```
15 def receive(stream):
16     line = stream.readline(MAX_BYTES + 1)
17     if not line:
18         raise ConnectionError("Conexão encerrada.")
19     if len(line) > MAX_BYTES or not line.endswith(b"\n"):
20         raise ValueError("Mensagem incompleta ou grande demais.")
21     message = json.loads(line)
22     if not isinstance(message, dict):
23         raise ValueError("A mensagem deve ser um objeto JSON.")
24     return message
```

readline acumula dados até encontrar a quebra de linha, o limite, o fim do fluxo ou uma falha. O limite é `MAX_BYTES + 1` para detectar se a mensagem ultrapassou o permitido. O código rejeita uma linha longa demais ou que terminou sem o delimitador.

Uma leitura vazia indica que a conexão chegou ao fim sem outra mensagem. Isso gera `ConnectionError`. `json.loads` interpreta o JSON recebido; ele aceita bytes nesse uso e reconstrói os valores. O código exige um dicionário, porque o protocolo espera um objeto com campos, e não uma lista ou um número solto.

Depois de um timeout ou erro de comunicação, o programa abandona o stream e a conexão por meio dos blocos `with`. Não tenta continuar lendo como se o conteúdo parcial fosse uma nova mensagem válida.

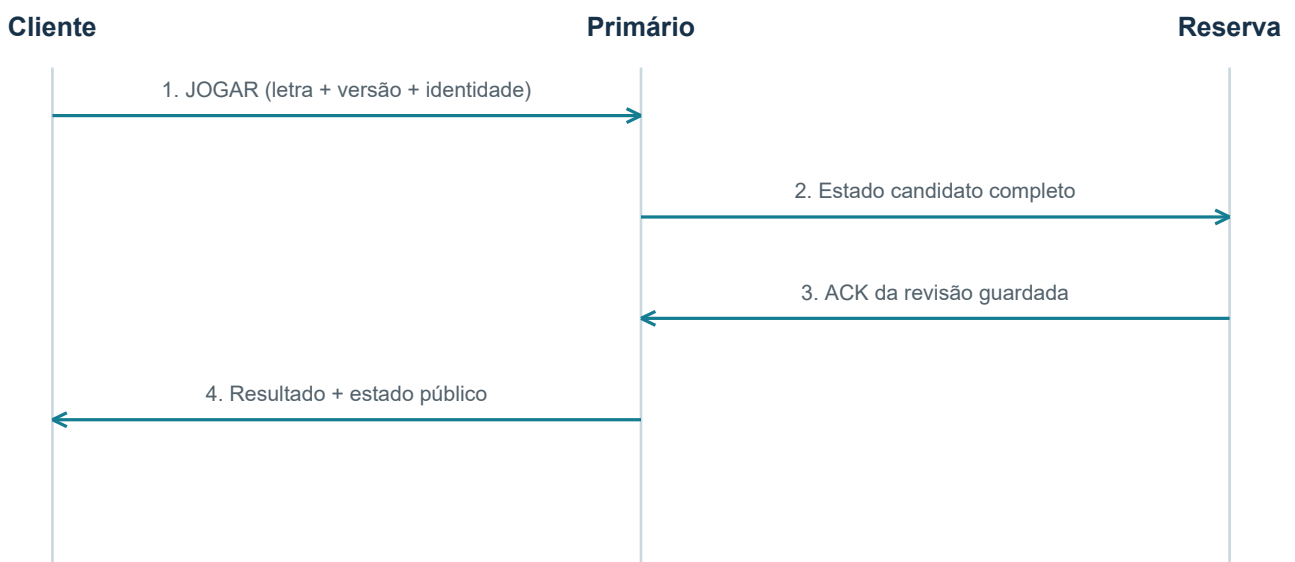
6.4 Por que não usar somente `recv(1024)`

Seria possível programar o enquadramento diretamente com `recv`, acumulando pedaços e procurando o delimitador. Porém, chamar `recv(1024)` uma única vez e imediatamente interpretar o resultado como JSON falharia quando a mensagem chegasse incompleta. O projeto usa `readline` para simplificar esse trabalho. O tamanho 1024, sozinho, não define uma mensagem.

Essa regra é usada nos dois canais: jogador-servidor e primário-reserva. O transporte é TCP em ambos; o formato é JSON em ambos; os tipos de mensagem e a duração da conexão são diferentes.

7. Uma jogada completa, do teclado aos dois terminais

Considere uma partida de Ana e Bruno cuja palavra é `SOCKET`. A palavra é conhecida pelo servidor; os clientes veem inicialmente _____. Ana joga primeiro. Ao digitar Z e pressionar Enter, ela produz uma tentativa incorreta.



O adversário recebe a atualização quando faz sua próxima consulta ESTADO.

1. `read_keyboard` lê o texto e o coloca em uma Queue, uma fila segura para troca de dados entre threads do cliente.

2. O laço run retira a entrada. parse a transforma em JOGAR e inclui a versão da sala conhecida naquele momento.
3. O cliente acrescenta token, deployment e request_id. Salva o comando em pending antes de enviá-lo.
4. exchange abre um socket até o servidor e envia o JSON terminado em \n.
5. Server.client lê o comando. Server.request entra na trava, identifica Ana e cria uma cópia candidata do estado.
6. apply encontra a sala; Room.guess verifica turno, versão e letra. Como Z não está na palavra, somente o contador de Ana aumenta, e a vez passa a Bruno.
7. O primário envia o estado candidato ao reserva. O reserva o guarda e responde com o ACK da revisão.
8. O primário adota a cópia candidata e responde a Ana. Ela limpa pending e desenha a resposta.
9. Bruno faz sua próxima consulta ESTADO e recebe a mesma evolução da partida. Seu terminal desenha o novo boneco de Ana e mantém seu próprio contador.

7.1 Exemplo de pedido

```
{
  "type": "JOGAR",
  "request_id": "7c9646e2-17ac-4e8b-8ed4-a6d57e979a75",
  "name": "Ana",
  "token": "token-ficticio-com-pelo-menos-32-caracteres",
  "deployment": "identificador-da-execucao",
  "letter": "Z",
  "version": 3
}
```

Este exemplo está quebrado em várias linhas apenas para leitura. Na rede, send produz uma linha. Os identificadores são ilustrativos: o cliente real usa os valores da sessão e a versão recebida do servidor. O servidor deriva a identidade do token; um campo name não permite escolher livremente quem será o autor da jogada.

7.2 Três resultados diferentes

ok indica o resultado da operação: uma regra violada pode retornar ok false com um code, como FORA_DA_VEZ. retry significa que aquele servidor não pode atender normalmente agora; o cliente tenta o próximo endereço e preserva o comando pendente. fatal indica que a sessão pertence a outra execução do sistema; o cliente mostra a orientação e encerra.

Uma letra errada dentro das regras retorna ok true e a mensagem Errou!. A operação foi aceita, embora a tentativa não tenha acertado a palavra. Isso evita confundir erro do jogador com erro de comunicação ou comando inválido.

7.3 Consultas periódicas e diferença entre telas

O cliente espera 0,5 segundo ao final do ciclo normal e depois consulta ou envia o próximo comando. Esse comportamento é chamado consulta periódica, ou polling. O intervalo real também inclui o tempo de rede e processamento. O servidor responde às consultas; não dispara espontaneamente atualizações para todos os terminais.

Portanto, os dois jogadores podem enxergar uma atualização em instantes ligeiramente diferentes. O estado vem do mesmo servidor, mas os terminais não são atualizados no mesmo microssegundo. room_version ajuda a rejeitar uma tentativa baseada em uma versão antiga.

8. Vários jogadores, salas e limite de duas pessoas

O servidor mantém um World, que reúne os jogadores, as salas, o identificador da execução e o número da próxima sala. Cada Room guarda seus próprios participantes, palavra, letras, erros, turno, versão e resultado. Os jogadores são relacionados à sala por `room_id`.

8.1 Algoritmo de entrada

Ao receber ENTRAR, apply valida o nome e recupera ou cria o jogador. enter procura outra sala AGUARDANDO com exatamente um participante que esteja online. Se encontrar, coloca o novo jogador nela. Se não encontrar, cria uma sala com o próximo número.

Entrada, com todos conectados	Sala	Situação
Ana	sala-1	Aguarda
Bruno	sala-1	A partida começa
Caio	sala-2	Aguarda
Dani	sala-2	A partida começa
Edu	sala-3	Aguarda

A busca não escolhe salas em andamento nem salas com dois jogadores. Além disso, Room.start exige exatamente dois participantes e status AGUARDANDO. Há, portanto, a escolha correta da vaga no lobby e uma verificação adicional ao iniciar a partida.

Trecho real: `forca/game.py` • `Room.start` • linha 58

```

58 def start(self, word: str):
59     if len(self.players) != 2 or self.status != "AGUARDANDO":
60         raise GameError("SALA_INVALIDA", "A sala precisa de dois jogadores para começar.")
61     if not re.fullmatch(r"[A-Z]+", word):
62         raise ValueError("A palavra deve conter apenas A a Z.")
63     self.word = word
64     self.errors = {p: 0 for p in self.players}
65     self.turn = self.players[0]
66     self.status = "EM_JOGO"
```

`random.choice` escolhe a palavra no primário quando o segundo jogador entra. O reserva recebe a palavra já sorteada dentro do estado completo. Ele não sorteia outra palavra para a mesma partida.

8.2 O que é a sala de espera

Uma sala com uma pessoa permanece AGUARDANDO. Não existe um servidor separado para a espera. O cliente continua consultando o estado, e o segundo participante inicia a partida. A ordem usual de chegada forma os pares quando os jogadores permanecem conectados; desconexões podem alterar qual sala tem um participante disponível.

Se Ana estava sozinha, desconecta, e Bruno entra enquanto ela está ausente, Bruno pode abrir outra sala. Quando Ana retorna e envia ENTRAR, enter pode juntá-la à sala de Bruno e cancelar a sala antiga com

motivo REAGRUPADA. O cliente tenta fazer isso automaticamente ao reconectar por uma consulta ESTADO em AGUARDANDO. O caso de retorno com comando pendente ainda tem uma limitação registrada em melhorias.md.

8.3 Várias salas não se misturam

As letras e os erros ficam em cada Room. Uma tentativa de Caio na sala-2 não altera a palavra da sala-1. O servidor usa o token para encontrar o jogador e sua associação à sala. O cliente não envia uma sala arbitrária para autorizar uma jogada.

Apesar dessa independência das regras, o processamento usa uma trava global: uma operação pode esperar por outra sala enquanto a replicação termina. Isso simplifica o projeto, mas limita o desempenho quando há muitas salas. Múltiplos jogadores atendidos não significam que todas as alterações acontecem ao mesmo tempo.

9. O servidor como semáforo: três mecanismos

O requisito “um jogador por vez” é aplicado pela combinação de uma trava de processamento e uma regra de turno. Há também um semáforo de capacidade. Compreender a diferença evita explicar o código de forma incorreta na apresentação.

Mecanismo	O que controla	Exemplo
threading.Lock	Uma operação por vez na região protegida do servidor	Dois ENTRAR não ocupam a mesma última vaga
Room.turn e check_turn	Qual participante pode jogar naquela sala	Bruno é recusado enquanto for a vez de Ana
BoundedSemaphore(64)	Quantas conexões estão em atendimento	A conexão excedente é fechada quando as 64 vagas estão ocupadas

9.1 A região protegida por Lock

Server.request usa with self.lock. Enquanto uma thread está nesse bloco, outra thread que precise da mesma trava espera. A sequência de validar, criar uma cópia, aplicar as regras, replicar e adotar o estado fica protegida contra outra requisição processada simultaneamente.

Sem a trava, duas entradas poderiam procurar uma vaga antes de qualquer uma registrar que a ocupou. Duas jogadas poderiam validar a mesma versão e depois sobrescrever alterações. A trava faz uma operação terminar sua etapa protegida antes de outra continuar.

A thread de sincronização também usa essa trava ao trocar mensagens com o reserva. Isso impede que um heartbeat e um estado sejam escritos e suas respostas lidas de forma intercalada por threads diferentes no mesmo stream. Threads atendem conexões concorrentemente, mas o trecho crítico fica serializado, ou seja, executado em sequência. A referência da biblioteca threading está em [R3].

9.2 A regra que autoriza o jogador

Trecho real: *forca/game.py* • *Room.check_turn* • linha 94

```
94 def check_turn(self, player_id: str, expected_version):
95     if player_id not in self.players:
96         raise GameError("NAO_PARTICIPANTE", "Você não participa desta sala.")
97     if self.status != "EM_JOGO":
98         raise GameError("PARTIDA_INDISPONIVEL", "A partida ainda não começou, está pausada ou ter
minou.")
99     if self.turn != player_id:
100         raise GameError("FORA_DA_VEZ", "Aguarde a sua vez.")
101     if type(expected_version) is not int or expected_version != self.version:
102         raise GameError("ESTADO_DESATUALIZADO", "A sala mudou. Confira o estado atual e tente nov
amente.")
```

As verificações seguem esta ordem: o jogador participa da sala; a partida está em andamento; é sua vez; a versão esperada é um inteiro igual à versão atual. Só depois a tentativa de letra ou palavra é avaliada.

É possível Bruno mandar bytes pela rede enquanto Ana joga. O socket não bloqueia o teclado dele por causa do turno. Quem rejeita a operação é a regra no servidor, retornando `FORA_DA_VEZ`. As consultas de estado continuam funcionando para ambos.

Se a tentativa é válida, acertar ou errar passa a vez, exceto quando a partida termina. Letra repetida, letra inválida, palavra repetida ou tentativa fora da vez não adicionam erro nem passam o turno. A validação local do cliente ajuda a pessoa, mas a validação decisiva é a do servidor.

9.3 Versão da sala e revisão global

`room_version` indica a versão daquela sala. O cliente a devolve em `JOGAR` ou `CHUTAR` para dizer sobre qual estado está tentando agir. Se outro evento mudou a sala, a tentativa antiga recebe `ESTADO_DESATUALIZADO`.

`revision` é o contador global usado na replicação. Pode crescer com entradas, saídas e resultados de comandos rejeitados. Não é igual ao total de letras jogadas e não deve ser confundido com `room_version`. O `ACK` do reserva confirma `revision`, não a versão de uma sala isolada.

10. Estados, erros e os dois bonecos

O servidor controla os números de erros e o resultado. O cliente recebe dados e transforma esses números em desenhos. Assim, o programa não precisa transportar uma imagem: basta transmitir o estado necessário para que ambos os clientes montem a mesma representação.

10.1 Estados da sala

Estado apresentado	Significado
AGUARDANDO	Há uma pessoa esperando o segundo jogador
EM_JOGO	A partida começou e os dois participantes são considerados presentes
PAUSADA	A sala está em andamento, mas há participante sem presença recente
ENCERRADA	Houve vencedor, por palavra completa, seis erros ou desistência
CANCELADA	Espera cancelada, abandono de nome ou reagrupamento

PAUSADA é calculado por `public_state` quando falta presença recente; não substitui o status interno `EM_JOGO` no objeto `Room`. O servidor considera online quem consultou nos últimos cinco segundos. Fechar um socket depois de uma consulta não desconecta o jogador logicamente: isso é normal, pois as conexões são curtas.

O mapa `seen` guarda os horários de presença no servidor que atende. Esses horários não são replicados. Depois da promoção, cada cliente volta a consultar, reconstruindo a presença. Até os dois retornarem, a partida pode aparecer pausada.

10.2 Erros separados

Trecho real: `forca/game.py` • `Room.conclude` • linha 104

```

104 def conclude(self, player_id: str, hit: bool, solved: bool):
105     if not hit:
106         self.errors[player_id] += 1
107     opponent = next(p for p in self.players if p != player_id)
108     if solved:
109         self.finish(player_id, "PALAVRA_COMPLETA")
110     elif self.errors[player_id] >= MAX_ERRORS:
111         self.finish(opponent, "SEIS_ERROS")
112     else:
113         self.turn = opponent
114     self.version += 1

```

`errors` é um dicionário por identificador de jogador. Quando `hit` é falso, somente `self.errors[player_id]` aumenta. Se a palavra foi resolvida, o autor vence. Se o autor chegou a seis erros, o adversário vence. Caso contrário, o turno passa ao adversário. A versão da sala aumenta após a tentativa válida.

O estado público contém uma lista de participantes, e cada item inclui `name`, `errors` e `connected`. Também inclui a palavra mascarada, as letras tentadas, os chutes errados e o turno. A palavra completa não é exposta como campo secreto ao cliente durante a partida; ela aparece na palavra mostrada depois do encerramento.

10.3 Do contador ao desenho

Trecho real: `forca/ui.py` • `gallows` • linha 15

```

15 def gallows(errors):
16     head = "O" if errors >= 1 else " "
17     trunk = "|" if errors >= 2 else " "
18     left = "/" if errors >= 3 else " "
19     right = "\\" if errors >= 4 else " "
20     leg_left = "/" if errors >= 5 else " "
21     leg_right = "\\" if errors >= 6 else " "
22     return [" +-----+", " |      |", f" {head}      |", f" {left}{trunk}{right}      |",
23            f" {leg_left} {leg_right}      |", "      |", " ====="]

```

Erros do jogador	Parte acrescentada
1	Cabeça
2	Tronco
3	Primeiro braço
4	Segundo braço
5	Primeira perna
6	Segunda perna; o jogador perde

`render` percorre os participantes e chama `gallows` para cada contador. Depois usa `zip` para colocar as linhas dos dois desenhos lado a lado. O identificador do cliente apenas determina qual nome recebe “(você)” e a frase “Sua vez”; os dois contadores continuam visíveis nos dois lados.

```

=====
FORCA | sala-1 | EM_JOGO

Ana (você)                                Bruno
+-----+                                +-----+
|      |                                |      |
O      |                                |      |
|      |                                |      |
|      |                                |      |
=====                                =====
Erros: 2/6                                Erros: 0/6

Palavra: S _ _ _ _
Letras tentadas: S, Z, X
Vez de Bruno. Aguarde.
=====

```

O exemplo acima foi produzido pela própria função `render`, com um estado didático: Ana tem dois erros e Bruno tem zero. No terminal de Bruno, os desenhos e números seriam os mesmos, mas “(você)” apareceria ao lado de Bruno. Não é uma captura de uma partida real.

11. Replicação: como a cópia chega à outra máquina

Replicar significa manter outra cópia dos dados. Neste projeto, ambas as cópias ficam em RAM, a memória usada pelo processo enquanto está rodando. JSON é o formato utilizado para transportar a cópia; não existe um arquivo JSON de partidas que os servidores compartilham por uma pasta de rede.

11.1 O que é copiado

O estado contém world, receipts e revision. world inclui jogadores, salas, palavras secretas, letras, chutes, erros, turnos, resultados e deployment_id. receipts guarda o último resultado de comando de cada jogador, usado em reenvios. revision identifica a revisão global da cópia.

Não se transfere a RAM inteira do computador. Não se copiam threads, sockets abertos, o sistema operacional nem a tela do terminal. O programa seleciona um dicionário com os dados necessários e o transmite. O reserva tem seu próprio processo Python e seus próprios recursos.

11.2 Conexão inicial do reserva

Server.synchronize escuta na porta 5001 do primário. Server.follow, no reserva, usa create_connection para chegar a essa porta. Portanto, nesse canal, o reserva desempenha o papel de quem inicia uma conexão, mesmo sendo chamado de servidor no projeto.

1. O reserva envia a chave compartilhada em um objeto com key.
2. O primário compara a chave com hmac.compare_digest e envia seu estado atual.
3. O reserva atribui a cópia recebida a self.state e envia ack com a revisão guardada.
4. O primário prossegue e envia uma segunda mensagem, normalmente um heartbeat.
5. Ao receber a segunda mensagem, o reserva passa a estar apto a assumir se a conexão falhar.

A segunda mensagem é importante: ela indica que o primário avançou além da espera pelo ACK inicial. Se o reserva recebesse a cópia e sumisse antes de concluir essa etapa, ele ainda não poderia se promover; o primário pode aguardar outra tentativa de sincronização inicial.

11.3 Cada alteração espera confirmação

Trecho real: servidor.py • Server.replicate • linha 44

```
44 def replicate(self, candidate):
45     # O reserva guarda a cópia ANTES de enviar ACK. Só então o jogador recebe sucesso.
46     send(self.backup, {"state": candidate})
47     ack = receive(self.backup)
48     if ack.get("ack") != candidate["revision"]:
49         raise ConnectionError("Confirmação de replicação inválida.")
```

A cópia candidata é enviada inteira. O reserva recebe a mensagem, guarda o estado e responde com sua revisão. Se o ACK não corresponder à revisão esperada, replicate levanta uma falha de comunicação.

```
Primário: deepcopy do estado atual
Primário: aplica o comando na cópia candidata
Primário -> reserva: {"state": candidato}
Reserva: self.state = estado recebido
Reserva -> primário: {"ack": revisão}
Primário: self.state = candidato
```

Primário -> cliente: resultado e estado público

deepcopy cria uma cópia independente das estruturas de dados. Assim, aplicar a regra no candidato não modifica imediatamente o estado que o primário adotou. Se a replicação falhar, ele pausa em vez de confirmar um estado sem a segunda cópia.

Essa replicação é chamada síncrona porque a resposta normal ao cliente espera a confirmação do reserva. Ela acrescenta tempo à jogada, mas permite que uma resposta confirmada antes da queda esteja na cópia do reserva. Depois da promoção, essa espera é desativada, pois não existe outro reserva conectado nesta versão.

11.4 Heartbeat

Heartbeat é uma mensagem pequena para verificar se a comunicação continua viva. O primário envia {"heartbeat": true}, recebe a resposta correspondente e espera 0,5 segundo para repetir. Isso ajuda a detectar uma falha mesmo quando nenhum jogador está enviando uma tentativa.

O canal é persistente: a mesma conexão transporta vários estados e heartbeats. No primário há timeout de três segundos nesse canal; no reserva, cinco segundos para receber. Fechar o processo pode gerar erro rapidamente; desligar fisicamente o computador pode exigir esperar o timeout.

12. O que acontece quando uma máquina cai

12.1 Queda do primário depois da sincronização

O reserva tenta ler a próxima mensagem e encontra conexão encerrada, erro de rede ou timeout. Como já recebeu a segunda mensagem da sincronização inicial, follow sai do ciclo de acompanhamento, marca promoted como verdadeiro e chama serve. A porta de jogadores passa a ser atendida no IP do próprio reserva.

Os clientes não recebem uma conexão TCP transplantada de uma VM para a outra. A conexão antiga falha. Cada cliente tenta outro endereço da lista --servers e abre uma nova conexão. Ele apresenta a mesma identidade de sessão, e o novo servidor encontra a sala no estado replicado.

O IP do primário também não é transferido. Não há IP virtual, proxy ou balanceador neste projeto. A troca acontece porque o cliente conhece os dois endereços e tenta o próximo ao detectar erro ou retry.

12.2 A resposta pode se perder depois de a jogada ser aplicada

Considere que o reserva já guardou a tentativa de Ana, mas o primário cai antes de a resposta chegar ao terminal dela. Ana não sabe se a tentativa valeu. Seu cliente preservou pending e reenvia o mesmo request_id ao reserva promovido. O recibo replicado permite reconhecer que a operação já aconteceu e devolver seu resultado, sem acrescentar outro erro nem passar a vez novamente.

O jogo também pode continuar com uma operação cujo retorno ficou incerto, pois o estado já havia chegado ao reserva. Por isso, “não recebi resposta” não significa automaticamente “nada aconteceu”. Reenvio e recibo resolvem essa dúvida no fluxo normal do cliente.

12.3 Matriz de falhas

Situação	Comportamento atual
Primário cai após sincronização	Reserva assume; clientes reconectam
Reserva cai	Primário se pausa; a partida não continua normalmente
Primário nunca sincronizou	Alterações ficam bloqueadas; reserva não assume sem estado confirmado inicialmente
Ambos encerram	Partidas em RAM são perdidas
Rede entre servidores falha	Primário deixa de confirmar alterações; reserva pode assumir por timeout
Cliente fecha	Após ausência de cinco segundos, a partida aparece pausada
Antigo primário volta	Não há reintegração automática ao novo ativo
Reserva promovido também cai	Não há outra cópia disponível nesta execução

Timeout é um sinal de ausência de comunicação, não uma prova de que o outro computador desligou. Para a demonstração, o cenário esperado é a parada do primário com o reserva e a rede funcionando. O desenho evita que o primário continue confirmando alterações sem ACK, mas não implementa eleição por maioria ou gerenciamento completo de reintegração.

Não basta religar o antigo primário para recuperar redundância. O reserva promovido não inicia o atendimento de um novo canal de replicação nessa versão. Para repetir o experimento, encerrem os dois servidores e iniciem uma nova execução sincronizada; os clientes então precisam de novas sessões. As partidas da execução anterior não são restauradas.

12.4 VM, container e computador físico

VirtualBox executa a VM; Docker executa o processo em um container dentro dela. Nenhum deles copia automaticamente as partidas deste programa para outro computador. A réplica do jogo é criada por `Server.replicate` e `Server.follow`.

Duas VMs no mesmo notebook compartilham a falha do notebook: se ele desligar, as duas param. Para o teste do professor, cada VM deve estar em um computador físico diferente. Os clientes que serão observados precisam ficar em um equipamento que permaneça ligado, assim como o roteador ou switch.

13. Sessões, identidade e comandos pendentes

O arquivo `sessions/ana.json` é do cliente. Ele guarda token, deployment e pending. Não contém a cópia oficial das partidas e não sincroniza a VM 1 com a VM 2. A sincronização das salas acontece no canal TCP entre os servidores.

Campo ou arquivo	Finalidade
token	Segredo aleatório que identifica o jogador ao servidor
player_id	Identificador obtido com SHA-256 do token
deployment	Identifica a execução dos servidores à qual a sessão pertence
pending	Um comando que ainda aguarda resposta definitiva
request_id	Identifica uma tentativa específica, inclusive em seus reenvios
.lock	Apoia a trava local que impede uso simultâneo do mesmo arquivo de sessão

13.1 Por que só o nome não basta

O nome é uma identificação para exibição; o token diferencia a sessão. Ao fechar e reabrir com o mesmo arquivo, o cliente usa o token antigo e pode retomar a sala. --nova-sessao gera outra identidade. Portanto, usar esse parâmetro sem querer pode impedir a retomada desejada ou resultar em NOME_EM_USO enquanto o jogador antigo continua ativo.

Nomes ativos são únicos em todas as salas, ignorando maiúsculas e espaços nas pontas. /sair libera o nome. Fora de partida em andamento, outro jogador pode reaproveitar um nome cujo dono ficou 180 segundos sem consultar. A liberação é avaliada quando outro ENTRAR solicita aquele nome; não há um serviço apagando nomes exatamente no segundo 180.

Na inicialização e após a promoção, since dá uma referência de tempo para quem ainda não consultou. Isso impede que a ausência de históricos de presença no reserva promova uma liberação imediata dos nomes.

13.2 Como pending protege uma tentativa

Antes de enviar, o cliente grava o comando completo em pending. O arquivo é escrito primeiro em um `.tmp` e depois substitui o anterior. Se a rede falhar, o mesmo comando permanece disponível. Se o cliente for reaberto com a mesma sessão, ele também pode tentar esse comando novamente.

O servidor guarda apenas o recibo do último comando de cada jogador. Esse desenho acompanha o cliente oficial, que mantém uma única operação pendente por vez. Não é uma promessa de reconhecer qualquer comando antigo reenviado fora de ordem por um cliente modificado.

Para comparar um reenvio, apply verifica `request_id` e uma impressão digital do conteúdo. Impressão digital é um resumo calculado por hash. Mesmo identificador com conteúdo diferente gera erro, porque não representa uma repetição válida. Quando uma operação recebeu um erro definitivo e a pessoa faz outra tentativa, o cliente gera um novo identificador.

13.3 Fechar e desistir são ações distintas

Ctrl+C fecha o cliente e conserva a sessão para retorno. /sair envia SAIR, encerra a participação e, durante uma partida, dá vitória ao adversário. Uma desconexão não equivale automaticamente a desistência. Se os dois servidores forem reiniciados, deployment muda; o arquivo antigo não recia a partida perdida em RAM.

Existe uma limitação conhecida: nomes como Ana e A-na podem resultar no mesmo nome de arquivo local, porque só caracteres alfanuméricos são usados. A opção --session permite escolher caminhos diferentes. A correção definitiva está registrada em melhorias.md.

14. Como executar com Python, VMs e Docker

14.1 Primeiro teste: tudo no mesmo computador

Execute na raiz do projeto, onde estão servidor.py e cliente.py. Abra quatro terminais. Este roteiro valida o fluxo básico, mas não comprova resistência ao desligamento desse computador.

Terminal 1:
python servidor.py

Terminal 2:
python servidor.py --modo reserva --port 5002

Terminal 3:
python cliente.py --name Ana --nova-sessao

Terminal 4:
python cliente.py --name Bruno --nova-sessao

Espere o log Reserva sincronizado. Jogadas liberadas. O cliente já usa por padrão 127.0.0.1:5000 e 127.0.0.1:5002. A porta 5002 evita conflito entre dois servidores de jogadores no mesmo endereço; o reserva continua conectando à porta de replicação 5001.

Em Windows, .\scripts\forca.ps1 servidor é um atalho executado a partir da raiz. Se o terminal já estiver dentro de scripts, o caminho é .\forca.ps1 servidor. No Ubuntu, use python3 quando esse for o nome do executável instalado.

14.2 Duas VMs em computadores distintos

Use os endereços a seguir apenas como exemplo: VM primária 192.168.1.10, VM reserva 192.168.1.20. Ambas devem estar acessíveis na mesma rede local. No VirtualBox, a configuração planejada é rede em modo Bridge: a VM participa da rede por um endereço próprio. Confirme seus endereços com ip -4 addr no Ubuntu.

Copie o projeto para cada VM. Instale Docker Engine e o plugin Compose para executar a entrega dockerizada. Confirme a instalação com docker --version e docker compose version. A instalação depende da distribuição; use as instruções oficiais correspondentes [R5]. Os clientes usam Python 3.12 ou superior e rodam fora dos containers no fluxo atual.

Na raiz de cada cópia, crie .env a partir de .env.example, por exemplo com cp .env.example .env no Linux. Ajuste os valores de acordo com a VM. A chave abaixo é ilustrativa; escolha o mesmo valor nas duas.

```
# .env da VM 1
MOD0=primario
PRIMARY_HOST=192.168.1.10
REPLICATION_KEY=chave-da-equipe
# .env da VM 2
```

```
MODO=reserva  
PRIMARY_HOST=192.168.1.10  
REPLICATION_KEY=chave-da-equipe
```

Execute `docker compose up --build` primeiro na VM primária e depois na reserva. Mantenha os logs visíveis. Os clientes devem usar os dois IPs reais:

```
python cliente.py --name Ana --servers 192.168.1.10:5000,192.168.1.20:5000  
python cliente.py --name Bruno --servers 192.168.1.10:5000,192.168.1.20:5000
```

Adicione `--nova-sessao` para começar uma nova identidade, inclusive depois de reiniciar os dois servidores. Para retomar uma partida da mesma execução, conserve a sessão existente e não acrescente esse parâmetro.

14.3 O papel do Dockerfile e do Compose

Dockerfile parte da imagem Python 3.12.13 slim, define `/app` como pasta de trabalho, copia o código e executa `python servidor.py`. Uma imagem é o pacote preparado para executar o programa; um container é uma execução dessa imagem. Não há instalação de pacotes de terceiros para o jogo.

EXPOSE 5000 5001 documenta as portas da imagem. É a seção `ports` do Compose que publica as portas na VM, associando 5000 a 5000 e 5001 a 5001. Publicar uma porta do container não abre automaticamente um firewall externo. No reserva, a porta 5000 só terá atendimento depois da promoção; publicar a porta não cria um servidor de jogadores antes disso.

O Compose passa `MODO`, `PRIMARY_HOST` e `REPLICATION_KEY` ao processo. `GAME_PORT` e `SYNC_PORT` são opções disponíveis no servidor, mas o Compose atual não as repassa e mantém o mapeamento fixo. Para mudar as portas na entrega Docker, seria necessário ajustar a configuração de ambiente e os mapeamentos de forma coerente.

`restart: "no"` evita reinício automático do antigo primário durante a demonstração. Reiniciar um container não restaura a RAM anterior nem o reintegra à partida recuperada. O mecanismo de continuidade é o reserva da aplicação.

14.4 Portas, firewall e internet

Na mesma rede local, não é necessário abrir encaminhamento de portas na internet do roteador. É necessário permitir que os dispositivos locais alcancem TCP 5000 nas duas VMs e que o reserva alcance TCP 5001 do primário. Firewalls e isolamento de dispositivos no Wi-Fi podem impedir essa comunicação.

No Windows, `Test-NetConnection 192.168.1.10 -Port 5000` verifica se a porta aceita conexão. No Ubuntu, `nc -vz 192.168.1.10 5001` pode ajudar, se `nc` estiver instalado. Esses testes verificam conexão, não validam o protocolo do jogo nem o ACK da replicação. Antes da promoção, é esperado que o reserva não atenda jogadores em 5000.

Prepare as imagens Docker antes da apresentação. Com imagens já construídas e os dispositivos na rede local, o jogo não depende de internet para comunicar clientes e servidores. Logs podem ser acompanhados com `docker compose logs -f servidor`; `docker compose down` encerra os containers e perde o estado em RAM daquela execução.

15. Testes e roteiro para apresentar ao professor

15.1 O que foi validado automaticamente

```
python -m unittest discover -s tests -v
```

Resultado desta documentação: 50 testes, OK, 8,382 segundos.

Ambiente: processos locais no Windows, comunicação TCP local.

Arquivo de testes	Quantidade	Exemplos cobertos
test_game.py	18	Turno, letras, acentos, chutes, vitória e seis erros
test_lobby.py	17	Salas, nomes, reagrupamento, pausa e reenvio
test_cliente.py	9	JSON por linha, comandos, tela e trava de sessão
test_servidor.py	6	Processos TCP reais, replicação, promoção e sincronização inicial

Os testes de servidor abrem portas locais escolhidas para a execução e encerram processos de teste. O teste de promoção compara o estado antes e depois de matar o primário e verifica um reenvio. Ele não representa desligamento elétrico, rede Bridge real nem falha em todas as janelas possíveis entre envio e confirmação.

O arquivo melhorias.md também identifica partes ainda sem cobertura automática completa, como o laço de teclado/reconexão do cliente, o caminho fatal de execução antiga e o limite de crescimento do estado. Ter 50 testes aprovados não significa ausência de qualquer defeito.

15.2 Roteiro de demonstração

1. Mostrem os dois computadores físicos, suas VMs e os containers. Expliquem qual é o primário e qual é o reserva.
2. Mostrem o log de sincronização e a lista dos dois endereços usada pelos clientes.
3. Abram Ana e Bruno: Ana espera, Bruno entra e a sala começa. Abram Caio e Dani: eles iniciam outra sala. Abram Edu: ele aguarda na terceira sala.
4. Enviem uma tentativa fora da vez. Mostrem que o servidor a rejeita e que erros e turno não são alterados por essa tentativa.
5. Façam um erro válido de Ana. Mostrem que só seu contador aumenta e que ambos os clientes desenham seu novo membro.
6. Registrem sala, palavra parcial, letras, erros, turno e versão antes da falha. Façam isso também na segunda sala.
7. Desliguem fisicamente o computador do primário. Mantenham clientes e rede no equipamento que fica ligado. Iniciem a medição do tempo.
8. Mostrem a promoção do reserva no log e a reconexão dos clientes. Confiram os dados registrados antes da queda e continuem jogando.
9. Abram um sexto jogador e verifiquem que encontra Edu esperando. Expliquem que agora há uma única cópia em execução.

10. Anotem o tempo e eventuais problemas. Para repetir, preparem uma nova execução dos dois servidores, com novas sessões.

15.3 Ficha de evidências para preencher

Item	Registro do grupo
Data e integrantes	_____
IPs das VMs e computadores usados	_____
Reserva sincronizado antes da queda	_____
Salas e estados antes da queda	_____
Estado correspondente após a troca	_____
Tempo observado até retomar jogadas	_____
Próxima jogada aceita no reserva	_____

Não há um resultado preenchido para o desligamento físico neste documento, porque esse teste não foi realizado durante sua geração. O objetivo de recuperação em até 15 segundos presente na especificação é uma meta a medir, não um valor garantido pelo código.

16. Dúvidas comuns e diagnóstico

16.1 “Fica aguardando primeira sincronização”

Significa que o reserva ainda não completou a etapa necessária para poder assumir. Verifiquem se o primário está executando, se PRIMARY_HOST aponta para o IP correto, se TCP 5001 está acessível e se as chaves são iguais. O primário pode estar atendendo 5000 e ainda não ter um reserva sincronizado; nesse caso, devolve retry aos comandos normais.

16.2 “O socket sabe quem é Ana?”

O socket conhece endereços e transporta bytes. A identidade do jogo vem do token no protocolo. Como o cliente abre conexões novas o tempo todo, usar a conexão como identidade permanente não serviria para retomar a mesma pessoa após uma queda.

16.3 “O JSON salva a partida?”

JSON é um formato. O projeto usa JSON tanto nas mensagens quanto no arquivo de sessão, com funções diferentes. O JSON enviado ao reserva carrega o estado das partidas; sessions guarda apenas os dados do cliente. Não há arquivo persistente de partidas no servidor. Se os dois processos perderem a RAM, os arquivos de sessão não reconstróem o jogo.

16.4 “Por que o reserva não atende antes da queda?”

Enquanto executa follow, o reserva está recebendo e guardando cópias. Ele só chama serve depois de perder o primário após a sincronização. Isso mantém um responsável pelo atendimento das partidas no fluxo normal.

16.5 “Posso usar a mesma porta nas duas máquinas?”

Sim. A combinação IP e porta é que determina o destino. 192.168.1.10:5000 e 192.168.1.20:5000 são destinos diferentes. No teste com os dois no mesmo endereço, o projeto usa 5000 e 5002 para os atendimentos de jogadores.

16.6 “Por que voltamos para outra sala?”

Confira se o cliente está usando o mesmo arquivo de sessão e se foi iniciado sem --nova-sessao. Confira também se houve /sair ou reinício dos dois servidores. A retomada depende do token e da execução ainda existente, não apenas do texto do nome. Jogadores esperando podem ser reagrupados em outra sala por uma regra prevista no lobby.

16.7 “O status passa sozinho para os clientes?”

O programa consulta automaticamente, mas o protocolo é de pedido e resposta. Cada cliente pede ESTADO periodicamente; o servidor responde com a sala. /estado, digitado pela pessoa, mostra o último estado recebido no cliente, enquanto as consultas automáticas continuam no laço normal.

16.8 Limites para manter o projeto compreensível

O estado inteiro é copiado e transmitido a cada comando processado. Salas encerradas e jogadores antigos ficam na memória, e a mensagem tem limite de 2 MB. Isso limita uma execução muito longa. O projeto privilegia uma demonstração pequena e legível, sem alegar capacidade para grandes quantidades de jogadores.

O tráfego não usa TLS, isto é, não tem a camada de criptografia normalmente associada a conexões seguras. A chave compartilhada protege de forma básica a entrada no canal de replicação, mas não cifra palavras ou tokens em trânsito. A documentação considera a rede local controlada da aula.

17. Glossário e mapa de leitura do código

Termo	Explicação no contexto do jogo
Socket	Ponta de comunicação usada por um processo
TCP	Transporte que oferece um fluxo ordenado de bytes por conexão
Protocolo	Regras de formato, mensagens e respostas combinadas pelos programas
Buffer	Área temporária em que bytes podem aguardar escrita ou leitura
Serialização	Conversão de dados para uma representação transmissível
Thread	Linha de execução dentro de um processo
Região crítica	Trecho que acessa dados compartilhados e precisa de proteção
Lock	Trava que permite uma thread por vez no trecho protegido
ACK	Confirmação; aqui, mensagem de que o reserva guardou certa revisão
Heartbeat	Mensagem periódica para acompanhar a comunicação
Replicação síncrona	Atualização que espera a confirmação da outra cópia
Promoção	Reserva passa a atender jogadores como ativo
Deduplicação	Reconhecer reenvio para não repetir a operação
Estado público	Dados da sala que o cliente pode receber para exibir o jogo
Failover	Troca do atendimento para o reserva após uma falha

17.1 Funções para mostrar na apresentação

Comecem por `cliente.exchange` e `Server.listen/serve` para demonstrar sockets reais. Mostrem `send` e `receive` em `wire.py` para explicar bytes, JSON e a quebra de linha. Em seguida, mostrem `Server.request` para ligar rede, regras e replicação.

Para as salas, mostrem `lobby.enter` e `Room.start`. Para “um por vez”, mostrem `self.lock` e `Room.check_turn`, deixando claro que a trava e o turno têm papéis diferentes. Para os bonecos, mostrem `Room.conclude`, `Room.public` e `ui.gallows/render`.

Para a queda do primário, mostrem `Server.replicate`, `Server.attend_backup`, `Server.follow` e o tratamento de falha no laço `cliente.run`. O apêndice mantém o código integral para acompanhar essa leitura sem depender de trechos isolados.

17.2 Exercícios de compreensão

1. Explique por que duas pessoas podem usar a porta 5000 sem compartilhar a mesma sessão.

2. Identifique onde a palavra é escondida do cliente, mas enviada ao reserva.
3. Descreva o que acontece se o ACK chega ao primário e a resposta ao jogador se perde.
4. Diferencie a fila de listen, a Queue do teclado e uma sala AGUARDANDO.
5. Explique por que duas VMs no mesmo computador não atendem ao teste de desligar esse computador.
6. Mostre o caminho de errors[player_id] até a cabeça desenhada nos dois terminais.

18. Referências e rastreabilidade

O código do próprio projeto é a fonte principal para todas as afirmações sobre comportamento, valores, regras e limitações. Os trechos e o apêndice foram extraídos dos arquivos da revisão 1ecf32097d7be882137d817587b803ad3b503e63. Consulte README.md, ARCHITECTURE.md, ROADMAP.md, melhorias.md e docs/protocolo-etapa-1.md para a documentação original do repositório.

As referências abaixo complementam os conceitos de biblioteca e infraestrutura. Elas não demonstram que o teste físico do grupo foi realizado. As APIs explicadas foram confrontadas com a documentação oficial durante a preparação deste guia.

- R1. Python - socket, interface de programação de rede. <https://docs.python.org/3/library/socket.html>
- R2. Python - Socket Programming HOWTO. <https://docs.python.org/3/howto/sockets.html>
- R3. Python - threading, threads e primitivas de sincronização. <https://docs.python.org/3/library/threading.html>
- R4. Docker - configuração de serviços no Compose. <https://docs.docker.com/reference/compose-file/services/>
- R5. Docker - instalação do Docker Engine no Ubuntu. <https://docs.docker.com/engine/install/ubuntu/>

Os exemplos usam APIs presentes no Python 3.12 ou superior, conforme o projeto. Os endereços IP apresentados são exemplos. A fonte editável deste documento acompanha o PDF em docs/documentacao-completa-sockets.md.

Apêndice A. Código integral da versão documentada

As páginas seguintes reproduzem o código do jogo, testes, scripts existentes e configurações de execução. Não incluem .env real, arquivos de sessão, dados pessoais, dependências instaladas nem arquivos internos do Git. Os números à esquerda são números de linha do arquivo original; linhas muito longas podem continuar visualmente na linha seguinte, sem alterar o conteúdo.

O script de geração da especificação anterior também aparece por fazer parte do projeto, embora não execute o jogo. Ele depende de ReportLab e fontes do Windows. O jogo e seus testes usam a biblioteca padrão. O apêndice documenta o estado atual dos arquivos e não modifica a implementação.

forca/wire.py

24 linhas • SHA-256: a5ee2aa19e1b2ca5...

```
1  """TCP transporta bytes: cada JSON termina com uma quebra de linha."""
2  import json
3
4  MAX_BYTES = 2_000_000
5
6
7  def send(stream, message):
8      data = json.dumps(message, ensure_ascii=False).encode("utf-8") + b"\n"
9      if len(data) > MAX_BYTES:
10         raise ValueError("Estado grande demais para o limite desta demonstração.")
11     stream.write(data)
12     stream.flush()
13
14
15  def receive(stream):
16     line = stream.readline(MAX_BYTES + 1)
17     if not line:
18         raise ConnectionError("Conexão encerrada.")
19     if len(line) > MAX_BYTES or not line.endswith(b"\n"):
20         raise ValueError("Mensagem incompleta ou grande demais.")
21     message = json.loads(line)
22     if not isinstance(message, dict):
23         raise ValueError("A mensagem deve ser um objeto JSON.")
24     return message
```

servidor.py

221 linhas • SHA-256: b59f64c690508377...

```

1  """Servidor ativo/reserva em RAM. Execute --help para ver as opções."""
2  import argparse
3  import copy
4  import hmac
5  import logging
6  import os
7  from pathlib import Path
8  import socket
9  import threading
10 import time
11
12 from forca.game import load_words
13 from forca.lobby import NAME_HOLD, apply, new_state, player_id, public_state
14 from forca.wire import receive, send
15
16 LOG = logging.getLogger("forca")
17 WORDS = Path(__file__).parent / "forca/words.txt"
18
19
20 class Server:
21     def __init__(self, host="0.0.0.0", port=5000, sync_port=5001,
22                 key="forca-aula", words=None):
23         self.host, self.port, self.sync_port = host, port, sync_port
24         self.key = key
25         self.words = load_words(words or WORDS.read_text(encoding="utf-8").splitlines())
26         self.state = new_state()
27         self.seen = {} # Presença local; uma conexão antiga não migra entre máquinas.
28         self.since = time.monotonic() # Início do atendimento; após a promoção, recomeça.
29         self.lock = threading.Lock() # Semáforo: validar + replicar é uma operação indivisível.
30         self.backup = None
31         self.promoted = False
32         self.frozen = False
33         self.slots = threading.BoundedSemaphore(64)
34
35     def online(self):
36         return {pid for pid, instant in self.seen.items() if time.monotonic() - instant < 5}
37
38     def idle(self):
39         """Jogadores sem consultar há NAME_HOLD segundos. Quem nunca consultou conta desde self.s
ince."""
40         now = time.monotonic()
41         return {pid for pid in self.state["world"]["players"]
42                 if now - self.seen.get(pid, self.since) >= NAME_HOLD}
43
44     def replicate(self, candidate):
45         # O reserva guarda a cópia ANTES de enviar ACK. Só então o jogador recebe sucesso.
46         send(self.backup, {"state": candidate})
47         ack = receive(self.backup)
48         if ack.get("ack") != candidate["revision"]:
49             raise ConnectionError("Confirmação de replicação inválida.")
50
51     def request(self, command):
52         with self.lock:
53             if command.get("type") == "PING":
54                 return {"ok": True, "role": "RESERVA_PROMOVIDO" if self.promoted else "PRIMARIO"}
55             if self.frozen or (not self.promoted and self.backup is None):
56                 return {"retry": True, "message": "Primário aguardando reserva ou pausado."}
57             pid = player_id(command.get("token"))
58             deployment = command.get("deployment")
59             if deployment and deployment != self.state["world"]["deployment_id"]:
60                 return {"fatal": True, "message": "Esta sessão pertence a outra execução. Use --n
ova-sessao."}
61             self.seen[pid] = time.monotonic()
62             result = {"ok": True}
63             if command.get("type") != "ESTADO":
64                 candidate = copy.deepcopy(self.state)
65                 result = apply(candidate, command, self.online(), self.words, self.idle())
66                 if not self.promoted:
67                     try:
68                         self.replicate(candidate)

```

```

69         except (OSError, ValueError):
70             self.frozen = True
71             LOG.error("Replicação perdida: primário PAUSADO. Reinicie como reserva ap
ós o teste.")
72             return {"retry": True, "message": "Replicação interrompida. Procurando re
serva."}
73         self.state = candidate
74         LOG.info("Estado %s confirmado (%s).", self.state["revision"], command["type"])
75         return {**result, "player_id": pid,
76                 "deployment": self.state["world"]["deployment_id"],
77                 "state": public_state(self.state, pid, self.online()),
78                 "role": "RESERVA_PROMOVIDO" if self.promoted else "PRIMARIO"}
79
80     def client(self, connection):
81         try:
82             with connection:
83                 connection.settimeout(4)
84                 with connection.makefile("rwb") as stream:
85                     try:
86                         response = self.request(receive(stream))
87                     except (ValueError, KeyError, TypeError) as exc:
88                         response = {"ok": False, "message": f"Comando inválido: {exc}"}
89                     except Exception: # Um comando inesperado não pode ficar sem resposta.
90                         LOG.exception("Erro inesperado ao atender um comando.")
91                         response = {"ok": False, "message": "Erro interno do servidor."}
92                     send(stream, response)
93         except (OSError, ValueError):
94             pass # O cliente reenviará a mesma tentativa ao reconectar.
95         finally:
96             self.slots.release()
97
98     def listen(self, port):
99         listener = socket.socket()
100         listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
101         listener.bind((self.host, port))
102         listener.listen(64)
103         return listener
104
105     def serve(self):
106         with self.listen(self.port) as listener:
107             LOG.info("Atendendo jogadores em %s:%s", self.host, self.port)
108             while True:
109                 connection, _ = listener.accept()
110                 if self.slots.acquire(blocking=False):
111                     threading.Thread(target=self.client, args=(connection,), daemon=True).start()
112                 else:
113                     connection.close()
114
115     def synchronize(self):
116         with self.listen(self.sync_port) as listener:
117             LOG.info("Aguardando reserva na porta %s", self.sync_port)
118             while True:
119                 connection, _ = listener.accept()
120                 try:
121                     with connection:
122                         connection.settimeout(3)
123                         with connection.makefile("rwb") as stream:
124                             self.attend_backup(stream)
125                 except (OSError, ValueError):
126                     pass # Fechar um canal rompido também falha ao descarregar o buffer.
127
128     def attend_backup(self, stream):
129         try:
130             hello = receive(stream)
131             # Em bytes: compare_digest recusa texto com acentos, o que derrubaria esta thread.
132             if not hmac.compare_digest(str(hello.get("key", "")).encode(), self.key.encode()):
133                 return
134             with self.lock:
135                 if self.frozen:
136                     return
137                 self.backup = stream
138             try:
139                 self.replicate(self.state)
140             except (OSError, ValueError):

```

```

141         # O reserva só assume depois da segunda mensagem (ver follow); sem ela, não h
142         # risco de dois ativos e o primário pode esperar outro reserva em vez de paus
143         ar.
144         self.backup = None
145         LOG.warning("Sincronização inicial com o reserva falhou; aguardando nova tent
146         ativa.")
147         return
148         LOG.info("Reserva sincronizado. Jogadas liberadas.")
149         while True:
150             with self.lock: # O primeiro heartbeat sai logo após o ACK: o reserva passa a po
151                 der assumir.
152                     if self.frozen:
153                         break
154                     send(stream, {"heartbeat": True})
155                     if receive(stream).get("heartbeat") is not True:
156                         raise ConnectionError("Heartbeat inválido.")
157                     time.sleep(0.5)
158             except (OSError, ValueError):
159                 with self.lock:
160                     if self.backup is stream:
161                         self.frozen = True
162                         LOG.warning("Liga  o com reserva perdida; prim  rio pausado.")
163             finally:
164                 with self.lock:
165                     if self.backup is stream:
166                         self.backup = None
167
168         def follow(self, primary):
169             synchronized = False
170             while not synchronized:
171                 try:
172                     with socket.create_connection((primary, self.sync_port), timeout=3) as connection
173                     :
174                         connection.settimeout(5)
175                         with connection.makefile("rwb") as stream:
176                             send(stream, {"key": self.key})
177                             first = True
178                             while True:
179                                 message = receive(stream)
180                                 # S   assume depois da segunda mensagem: ela prova que o prim  rio rece
181                                 beu
182                                 # o ACK da c  pia inicial e n  o vai procurar outro reserva.
183                                 synchronized = synchronized or not first
184                                 first = False
185                                 if "state" in message:
186                                     self.state = message["state"]
187                                     send(stream, {"ack": self.state["revision"]})
188                                 else:
189                                     send(stream, {"heartbeat": True})
190                             except (OSError, ValueError):
191                                 if not synchronized:
192                                     LOG.info("Aguardando primeira sincroniza  o com %s...", primary)
193                                     time.sleep(1)
194                                 self.promoted = True
195                                 self.since = time.monotonic() # Os jogadores ainda v  o reconectar: ningu  m perde o nome
196                                 j  .
197                                 LOG.warning("PRIM  RIO CAIU: reserva assumiu na revis  o %s. Agora h   apenas uma c  pia.",
198                                     self.state["revision"])
199                                 self.serve()
200
201         def main():
202             parser = argparse.ArgumentParser(description=__doc__)
203             parser.add_argument("--modo", choices=["prim  rio", "reserva"], default=os.getenv("MOD0", "pri
204             m  rio"))
205             parser.add_argument("--host", default="0.0.0.0")
206             parser.add_argument("--port", type=int, default=int(os.getenv("GAME_PORT", "5000")))
207             parser.add_argument("--sync-port", type=int, default=int(os.getenv("SYNC_PORT", "5001")))
208             parser.add_argument("--prim  rio", default=os.getenv("PRIMARY_HOST", "127.0.0.1"))
209             parser.add_argument("--key", default=os.getenv("REPLICATION_KEY", "forca-aula"))
210             args = parser.parse_args()
211             logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s", datefmt="%H:%M:%S")
212             try:

```

```

207         server = Server(args.host, args.port, args.sync_port, args.key)
208     except ValueError as exc:
209         raise SystemExit(f"Lista de palavras inválida ({WORDS.name}): {exc}")
210     try:
211         if args.modo == "reserva":
212             server.follow(args.primario)
213         else:
214             threading.Thread(target=server.synchronize, daemon=True).start()
215             server.serve()
216     except KeyboardInterrupt:
217         print("\nServidor encerrado.")
218
219
220 if __name__ == "__main__":
221     main()

```

cliente.py

194 linhas • SHA-256: *ec9af9bc78f0cc3b...*

```

1  """Terminal com reconexão: a sessão identifica o jogador mesmo após a queda."""
2  import argparse
3  import json
4  import os
5  from pathlib import Path
6  import queue
7  import secrets
8  import socket
9  import threading
10 import time
11 import uuid
12
13 try:
14     import msvcrt
15 except ImportError:
16     import fcntl
17
18 from forca.game import normalize
19 from forca.ui import HELP, render
20 from forca.wire import receive, send
21
22
23 def exchange(address, command):
24     host, port = address.rsplit(":", 1)
25     with socket.create_connection((host, int(port)), timeout=2) as connection:
26         connection.settimeout(5)
27         with connection.makefile("rwb") as stream:
28             send(stream, command)
29             return receive(stream)
30
31
32 def read_keyboard(commands):
33     while True:
34         try:
35             commands.put(input().strip())
36         except (EOFError, KeyboardInterrupt):
37             commands.put("/sair")
38     return
39
40
41 def save(path, session):
42     path.parent.mkdir(parents=True, exist_ok=True)
43     temporary = path.with_suffix(".tmp")
44     temporary.write_text(json.dumps(session), encoding="utf-8")
45     temporary.replace(path)
46
47
48 def lock(path):
49     """Impede dois clientes abertos com o mesmo arquivo de sessão. O sistema libera ao encerrar."
50     """
51     path.parent.mkdir(parents=True, exist_ok=True)

```

```

51     handle = open(path.with_suffix(".lock"), "a+b")
52     handle.seek(0)
53     try:
54         if os.name == "nt":
55             msvcrt.locking(handle.fileno(), msvcrt.LK_NBLCK, 1)
56         else:
57             fcntl.flock(handle, fcntl.LOCK_EX | fcntl.LOCK_NB)
58     except OSError:
59         handle.close()
60         return None
61     return handle
62
63
64 def parse(action, state):
65     """Converte o que foi digitado em comando para o servidor, ou em um aviso local (str)."""
66     name, _, argument = action.partition(" ")
67     name = name.lower()
68     version = state["room_version"] if state else None
69     if name == "/nova":
70         return {"type": "ENTRAR"}
71     if name == "/sair":
72         return {"type": "SAIR"}
73     if name == "/chute":
74         if not argument.strip():
75             return "Use /chute seguido da palavra. Exemplo: /chute SOCKET"
76         return {"type": "CHUTAR", "word": argument.strip(), "version": version}
77     if name.startswith("/"):
78         return "Comando desconhecido. Digite /ajuda para ver os comandos."
79     if len(normalize(action)) != 1: # "ç" e "ã" contam como uma letra.
80         return "Digite uma única letra, ou /chute seguido da palavra para tentar a palavra inteira."
81     return {"type": "JOGAR", "letter": action, "version": version}
82
83
84 def run(name, servers, path, fresh=False):
85     guard = lock(path)
86     if guard is None:
87         print(f"Já existe um cliente aberto com a sessão {path}. Use outro --name ou --session.")
88         return
89     previous = path.read_text(encoding="utf-8") if path.exists() else None
90     session = json.loads(previous) if previous and not fresh else {
91         "token": secrets.token_urlsafe(32), "deployment": None, "pending": None}
92     save(path, session)
93     commands = queue.Queue()
94     threading.Thread(target=read_keyboard, args=(commands,), daemon=True).start()
95     state, me, last_screen, current_server = None, None, None, 0
96     connected = False
97     enter = session["deployment"] is None
98     backlog = [] # Comandos para o servidor digitados enquanto outro ainda espera resposta.
99     print("Conectando... Digite /ajuda para ver os comandos.")
100    while True:
101        # Comandos locais respondem na hora, mesmo sem conexão; os do servidor esperam a vez.
102        while not commands.empty():
103            action = commands.get()
104            keyword = action.split(" ", 1)[0].lower()
105            if not action:
106                continue
107            if keyword in {"/ajuda", "/help"}:
108                print(HELP)
109            elif keyword == "/estado" and not connected:
110                print("Sem conexão com um servidor no momento. Tentando reconectar..."
111                      + (" Último estado conhecido:" if state else ""))
112                if state:
113                    print(render(state, me))
114            elif keyword == "/estado":
115                last_screen = render(state, me)
116                print(last_screen)
117            else:
118                parsed = parse(action, state)
119                if isinstance(parsed, str):
120                    print(parsed) # Aviso local: comando desconhecido, /chute vazio etc.
121                else:
122                    backlog.append(action)
123            if not session["pending"] and (enter or backlog):

```

```

124         action = "/nova" if enter else backlog.pop(0)
125         enter = False
126         command = parse(action, state) # Versão da sala atual, não a do momento em que foi d
igitado.
127         session["pending"] = {
128             **command, "request_id": str(uuid.uuid4()), "name": name,
129             "token": session["token"], "deployment": session["deployment"]}
130         save(path, session) # O reenvio após uma queda mantém este identificador.
131         command = session["pending"] or {"type": "ESTADO", "token": session["token"],
132             "deployment": session["deployment"]}
133         address = servers[current_server]
134         try:
135             response = exchange(address, command)
136             if response.get("fatal"):
137                 print(response["message"])
138                 return
139             if response.get("retry"):
140                 raise ConnectionError(response["message"])
141         except (OSError, ValueError):
142             if connected:
143                 print("Conexão perdida. Procurando outro servidor...")
144                 connected = False
145                 current_server = (current_server + 1) % len(servers)
146                 time.sleep(0.5)
147                 continue
148             reconnected = not connected
149             if reconnected:
150                 print(f"Conectado a {address} ({response.get('role', '')}).")
151                 connected = True
152             session["deployment"] = response.get("deployment", session["deployment"])
153             if session["pending"]:
154                 print(response.get("message", ""))
155                 kind = session["pending"]["type"]
156                 leaving = kind == "SAIR" and (response.get("ok") or response.get("code") == "SEM_SALA
")
157             session["pending"] = None
158             save(path, session)
159             if response.get("code") == "NOME_EM_USO":
160                 if fresh and previous: # Devolve a sessão anterior deste nome, que ainda pode se
r retomada.
161                     path.write_text(previous, encoding="utf-8")
162                 elif fresh:
163                     path.unlink()
164                 print("Abra o cliente com outro --name. Se o nome for seu, retome sem --nova-sess
ao.")
165             return
166             if leaving:
167                 return
168             state, me = response.get("state"), response.get("player_id")
169             if reconnected and command["type"] == "ESTADO" and state and state["status"] == "AGUARDAN
DO":
170                 enter = True # Ao voltar, procura quem esteja esperando em outra sala.
171             screen = render(state, me)
172             if screen != last_screen:
173                 print(screen)
174                 last_screen = screen
175             time.sleep(0.5) # Atualiza os dois bonecos nos dois terminais.
176
177
178 def main():
179     parser = argparse.ArgumentParser(description=__doc__)
180     parser.add_argument("--name", required=True)
181     parser.add_argument("--servers", default=os.getenv("CLIENT_SERVERS", "127.0.0.1:5000,127.0.0.
1:5002"))
182     parser.add_argument("--session", help="Arquivo individual de sessão deste jogador.")
183     parser.add_argument("--nova-sessao", action="store_true")
184     args = parser.parse_args()
185     filename = "".join(c for c in args.name.casefold() if c.isalnum()) or "jogador"
186     path = Path(args.session or f"sessions/{filename}.json")
187     try:
188         run(args.name, [s.strip() for s in args.servers.split(",")], path, args.nova_sessao)
189     except KeyboardInterrupt:
190         print("\nCliente fechado. A sessão foi guardada para reconexão.")
191

```

```

192
193 if __name__ == "__main__":
194     main()

```

forca/game.py

152 linhas • SHA-256: 2d8f6f4a494aa428...

```

1  """Regras de uma partida: independentes da rede e da replicação."""
2  from dataclasses import asdict, dataclass, field
3  import re
4  import unicodedata
5
6  FINISHED = {"ENCERRADA", "CANCELADA"}
7  MAX_ERRORS = 6
8  MAX_WORD = 30 # O estado inteiro é replicado a cada jogada; chutes longos não fazem sentido.
9
10
11 class GameError(Exception):
12     def __init__(self, code: str, message: str):
13         super().__init__(message)
14         self.code = code
15
16
17 def normalize(text: str) -> str:
18     """Maiúsculas sem acentos: 'ç' vira 'C' e 'CONEXÃO' vira 'CONEXAO'."""
19     decomposed = unicodedata.normalize("NFKD", text)
20     return "".join(c for c in decomposed if not unicodedata.combining(c)).upper()
21
22
23 def load_words(lines) -> list[str]:
24     """Normaliza a lista de palavras e recusa, já na partida do servidor, o que não for só letras
25     ."""
26     words = [normalize(line.strip()) for line in lines if line.strip()]
27     invalid = [w for w in words if not re.fullmatch(rf"[A-Z]{{1,{MAX_WORD}}}", w)]
28     if invalid:
29         raise ValueError(f"use só letras, até {MAX_WORD} por palavra, sem espaços ou hífen: "
30             + ", ".join(invalid))
31     if not words:
32         raise ValueError("a lista está vazia.")
33     return words
34
35 @dataclass
36 class Player:
37     id: str
38     name: str
39     token_hash: str
40     room_id: str | None = None
41     active: bool = True # Só jogadores ativos reservam o nome; /sair libera.
42
43
44 @dataclass
45 class Room:
46     id: str
47     players: list[str] = field(default_factory=list)
48     word: str = ""
49     guesses: list[str] = field(default_factory=list)
50     wrong_words: list[str] = field(default_factory=list)
51     errors: dict[str, int] = field(default_factory=dict)
52     turn: str | None = None
53     status: str = "AGUARDANDO"
54     version: int = 1
55     winner: str | None = None
56     reason: str | None = None
57
58     def start(self, word: str):
59         if len(self.players) != 2 or self.status != "AGUARDANDO":
60             raise GameError("SALA_INVALIDA", "A sala precisa de dois jogadores para começar.")
61         if not re.fullmatch(r"[A-Z]+", word):
62             raise ValueError("A palavra deve conter apenas A a Z.")

```

```

63         self.word = word
64         self.errors = {p: 0 for p in self.players}
65         self.turn = self.players[0]
66         self.status = "EM_JOGO"
67
68     def guess(self, player_id: str, letter, expected_version) -> bool:
69         self.check_turn(player_id, expected_version)
70         if not isinstance(letter, str) or not re.fullmatch(r"[A-Z]", normalize(letter)):
71             raise GameError("LETRA_INVALIDA", "Digite uma única letra de A a Z.")
72         letter = normalize(letter)
73         if letter in self.guesses:
74             raise GameError("LETRA_REPETIDA", "Essa letra já foi tentada na sala.")
75         self.guesses.append(letter)
76         hit = letter in self.word
77         self.conclude(player_id, hit, solved=set(self.word) <= set(self.guesses))
78         return hit
79
80     def guess_word(self, player_id: str, word, expected_version) -> bool:
81         """Chute da palavra inteira: acerto vence; erro custa um membro e passa a vez."""
82         self.check_turn(player_id, expected_version)
83         if not isinstance(word, str) or not re.fullmatch(rf"[A-Z]{{{1},{MAX_WORD}}}", normalize(word.strip())):
84             raise GameError("PALAVRA_INVALIDA", f"Chute uma única palavra de até {MAX_WORD} letras, sem espaços.")
85         word = normalize(word.strip())
86         if word in self.wrong_words:
87             raise GameError("PALAVRA_REPETIDA", "Essa palavra já foi chutada na sala.")
88         hit = word == self.word
89         if not hit:
90             self.wrong_words.append(word)
91         self.conclude(player_id, hit, solved=hit)
92         return hit
93
94     def check_turn(self, player_id: str, expected_version):
95         if player_id not in self.players:
96             raise GameError("NAO_PARTICIPANTE", "Você não participa desta sala.")
97         if self.status != "EM_JOGO":
98             raise GameError("PARTIDA_INDISPONIVEL", "A partida ainda não começou, está pausada ou terminou.")
99         if self.turn != player_id:
100             raise GameError("FORA_DA_VEZ", "Aguarde a sua vez.")
101         if type(expected_version) is not int or expected_version != self.version:
102             raise GameError("ESTADO_DESATUALIZADO", "A sala mudou. Confira o estado atual e tente novamente.")
103
104     def conclude(self, player_id: str, hit: bool, solved: bool):
105         if not hit:
106             self.errors[player_id] += 1
107         opponent = next(p for p in self.players if p != player_id)
108         if solved:
109             self.finish(player_id, "PALAVRA_COMPLETA")
110         elif self.errors[player_id] >= MAX_ERRORS:
111             self.finish(opponent, "SEIS_ERROS")
112         else:
113             self.turn = opponent
114             self.version += 1
115
116     def finish(self, winner: str | None, reason: str):
117         self.status = "ENCERRADA" if winner else "CANCELADA"
118         self.winner = winner
119         self.reason = reason
120         self.turn = None
121
122     def public(self, players: dict[str, Player], online: set[str]) -> dict:
123         return {
124             "room_id": self.id, "room_version": self.version,
125             "status": self.status,
126             "players": [{"player_id": pid, "name": players[pid].name,
127                         "errors": self.errors.get(pid, 0), "connected": pid in online}
128                        for pid in self.players],
129             "masked_word": "".join(c if c in self.guesses or self.status in FINISHED else "_"
130                                   for c in self.word),
131             "guesses": list(self.guesses), "wrong_words": list(self.wrong_words), "turn": self.turn,

```

```

132         "winner": self.winner, "reason": self.reason, "max_errors": MAX_ERRORS,
133     }
134
135
136 @dataclass
137 class World:
138     deployment_id: str
139     players: dict[str, Player] = field(default_factory=dict)
140     rooms: dict[str, Room] = field(default_factory=dict)
141     next_room: int = 1
142
143     def to_dict(self) -> dict:
144         return asdict(self)
145
146     @classmethod
147     def from_dict(cls, data: dict):
148         return cls(deployment_id=data["deployment_id"],
149                   players={k: Player(**v) for k, v in data["players"].items()},
150                   rooms={k: Room(**v) for k, v in data["rooms"].items()},
151                   next_room=data["next_room"])
152

```

forca/lobby.py

141 linhas • SHA-256: d6e905d8aa6bf534...

```

1  """Salas e sessões. Não abre sockets: o servidor replica o resultado destas regras."""
2  import hashlib
3  import random
4  import uuid
5
6  from .game import FINISHED, GameError, Player, Room, World
7
8  NAME_HOLD = 180 # Segundos sem consultar até o nome de quem não está em partida poder ser reapro
   veitado.
9
10
11 def player_id(token):
12     if not isinstance(token, str) or not 32 <= len(token) <= 128:
13         raise ValueError("Token de sessão inválido.")
14     return hashlib.sha256(token.encode()).hexdigest()
15
16
17 def new_state():
18     return {"world": World(str(uuid.uuid4())).to_dict(), "receipts": {}, "revision": 0}
19
20
21 def public_state(state, pid, online):
22     world = World.from_dict(state["world"])
23     player = world.players.get(pid)
24     room = world.rooms.get(player.room_id) if player else None
25     result = room.public(world.players, online) if room else None
26     if result and room.status == "EM_JOGO" and not set(room.players) <= online:
27         result["status"] = "PAUSADA"
28     return result
29
30
31 def claim_name(world, pid, name, idle):
32     """Recusa nome em uso. Quem sumiu há NAME_HOLD segundos e não está em partida perde o nome."""
33     for other in world.players.values():
34         if other.id == pid or not other.active or other.name.casefold() != name.casefold():
35             continue
36         room = world.rooms.get(other.room_id)
37         if other.id not in idle or (room and room.status == "EM_JOGO"):
38             raise GameError("NOME_EM_USO", f"O nome {name} já está em uso. Escolha outro nome.")
39         if room and room.status == "AGUARDANDO":
40             room.finish(None, "ABANDONO")
41             room.version += 1
42         other.room_id, other.active = None, False
43

```

```

44
45 def enter(world, player, online, words):
46     """Coloca o jogador em uma sala e devolve a mensagem de resposta."""
47     room = world.rooms.get(player.room_id)
48     if room and room.status == "EM_JOGO":
49         return f"Você já está em uma partida na {room.id}. Use /sair para desistir."
50     waiting = room is not None and room.status == "AGUARDANDO"
51     partner = next((r for r in world.rooms.values()
52                     if r is not room and r.status == "AGUARDANDO" and len(r.players) == 1
53                     and r.players[0] in online), None)
54     if waiting and partner is None:
55         return f"Você continua aguardando na {room.id}."
56     if waiting: # Dois jogadores esperando em salas separadas: junta os dois na sala do outro.
57         room.finish(None, "REAGRUPADA")
58         room.version += 1
59     room = partner
60     if room is None:
61         room = Room(f"sala-{world.next_room}")
62         world.next_room += 1
63         world.rooms[room.id] = room
64     room.players.append(player.id)
65     room.errors[player.id] = 0
66     player.room_id = room.id
67     if len(room.players) == 2:
68         room.start(random.choice(words))
69     room.version += 1
70     return f"Você está na {room.id}."
71
72
73 def apply(state, command, online, words, idle=frozenset()):
74     """Modifica uma cópia do estado. Reenvios devolvem o resultado já registrado.
75
76     online: jogadores que consultaram nos últimos segundos.
77     idle: jogadores sem consultar há NAME_HOLD segundos, cujo nome pode ser liberado.
78
79     pid = player_id(command.get("token"))
80     request_id = command.get("request_id")
81     if not isinstance(request_id, str):
82         raise ValueError("request_id deve ser um UUID em texto.")
83     request_id = str(uuid.UUID(request_id))
84     fingerprint = hashlib.sha256(str(sorted(command.items())).encode()).hexdigest()
85     previous = state["receipts"].get(pid)
86     if previous and previous["request_id"] == request_id:
87         if previous["fingerprint"] != fingerprint:
88             raise ValueError("Identificador reutilizado para outro comando.")
89     return previous["result"]
90
91     world = World.from_dict(state["world"])
92     kind = command.get("type")
93     try:
94         if kind == "ENTRAR":
95             name = command.get("name", "")
96             if not isinstance(name, str) or not 1 <= len(name.strip()) <= 24 or not name.isprinta
97 ble():
98             raise GameError("NOME_INVALIDO", "Use um nome de 1 a 24 caracteres.")
99             name = name.strip()
100             player = world.players.get(pid)
101             if player is None or not player.active:
102                 # Nomes são únicos entre jogadores ativos de todas as salas, sem diferenciar maiú
103 sculas.
104                 claim_name(world, pid, name, idle)
105                 player = world.players.setdefault(pid, Player(pid, name, pid))
106                 player.name, player.active = name, True
107                 result = {"ok": True, "message": enter(world, player, online, words)}
108             elif kind in {"JOGAR", "CHUTAR"}:
109                 player = world.players.get(pid)
110                 room = world.rooms.get(player.room_id) if player else None
111                 if room is None:
112                     raise GameError("SEM_SALA", "Entre em uma sala com /nova.")
113                 if not set(room.players) <= online:
114                     raise GameError("PAUSADA", "Aguardando o outro jogador reconectar.")
115                 if kind == "JOGAR":
116                     hit = room.guess(pid, command.get("letter"), command.get("version"))
117                     result = {"ok": True, "message": "Acertou!" if hit else "Errou!"}

```

```

116         else:
117             hit = room.guess_word(pid, command.get("word"), command.get("version"))
118             result = {"ok": True, "message": "Acertou a palavra!" if hit
119                     else "A palavra não coincide. Você perdeu um membro e a vez passou."}
120         elif kind == "SAIR":
121             player = world.players.get(pid)
122             if player is None or not player.active:
123                 raise GameError("SEM_SALA", "Você não está em nenhuma sala.")
124             room = world.rooms.get(player.room_id)
125             if room and room.status not in FINISHED:
126                 opponent = next((p for p in room.players if p != pid), None)
127                 room.finish(opponent, "DESISTENCIA")
128                 room.version += 1
129                 player.room_id, player.active = None, False
130                 result = {"ok": True, "message": "Você saiu."}
131         else:
132             raise GameError("COMANDO_INVALIDO", "Comando desconhecido.")
133     except GameError as exc:
134         result = {"ok": False, "message": str(exc), "code": exc.code}
135     state["world"] = world.to_dict()
136     if pid in world.players:
137         # O cliente tem um único comando pendente por vez: basta o último recibo de cada jogador.
138         # Comandos de quem nunca entrou não mudam nada e não ocupam espaço no estado replicado.
139         state["receipts"][pid] = {"request_id": request_id, "fingerprint": fingerprint, "result":
result}
140     state["revision"] += 1
141     return result

```

forca/ui.py

82 linhas • SHA-256: 8d1e6bd559a6704b...

```

1  """Apresentação textual: os dois bonecos são desenhados em toda atualização."""
2
3  HELP = "\n".join([
4      "Comandos:",
5      "  <letra>      Tenta uma letra na sua vez",
6      "  /chute <palavra>  Tenta adivinhar a palavra inteira na sua vez",
7      "  /estado        Mostra a partida: forcas, palavra e letras tentadas",
8      "  /nova          Entra em uma nova partida depois que a atual termina",
9      "  /sair          Desiste da partida e fecha o cliente",
10     "  /ajuda ou /help  Mostra esta lista",
11     "  Ctrl+C          Fecha o cliente e guarda a sessão para retomar depois",
12 ])
13
14
15 def gallows(errors):
16     head = "O" if errors >= 1 else " "
17     trunk = "|" if errors >= 2 else " "
18     left = "/" if errors >= 3 else " "
19     right = "\"\" if errors >= 4 else " "
20     leg_left = "/" if errors >= 5 else " "
21     leg_right = "\"\" if errors >= 6 else " "
22     return [" +-----+", " |      |", f" {head}      |", f" {left}{trunk}{right}      |",
23            f" {leg_left} {leg_right}      |", "      |", " ====="]
24
25
26 def outcome(state, player_id):
27     """Resultado do ponto de vista de quem olha, sem os códigos internos do protocolo."""
28     winner = state["winner"]
29     if winner is None:
30         return "Partida cancelada."
31     names = {p["player_id"]: p["name"] for p in state["players"]}
32     loser = next(pid for pid in names if pid != winner)
33     if winner == player_id:
34         details = {"PALAVRA_COMPLETA": "Você completou a palavra.",
35                   "SEIS_ERROS": f"{names[loser]} chegou a seis erros.",
36                   "DESISTENCIA": f"{names[loser]} desistiu."}
37     return "Você venceu! " + details.get(state["reason"], "")
38     details = {"PALAVRA_COMPLETA": f"{names[winner]} completou a palavra.",
39               "SEIS_ERROS": "Você chegou a seis erros.",

```

```

40         "DESISTENCIA": "Você desistiu."}
41     return f"Você perdeu. {details.get(state['reason'], '')}"
42
43
44 def render(state, player_id):
45     if not state:
46         return "Você não está em nenhuma sala. Digite /nova para entrar em uma."
47     players = state["players"]
48     lines = ["=" * 68, f"FORÇA | {state['room_id']} | {state['status']}", ""]
49     labels = []
50     drawings = []
51     for p in players:
52         labels.append(f"{p['name']}{' (você)' if p['player_id'] == player_id else ''}")
53         drawings.append(gallows(p["errors"]))
54     if len(players) == 1:
55         labels.append("Aguardando adversário")
56         drawings.append(gallows(0))
57     lines.append(f"{labels[0]:<34}{labels[1]}")
58     for left, right in zip(*drawings):
59         lines.append(f"{left:<34}{right}")
60     left_errors = players[0]["errors"]
61     right_errors = players[1]["errors"] if len(players) == 2 else 0
62     lines.extend([f"Erros: {left_errors}/6{' ' * 24}Erros: {right_errors}/6", ""])
63     if state["masked_word"]:
64         lines.append("Palavra: " + " ".join(state["masked_word"]))
65     lines.append("Letras tentadas: " + (" ".join(state["guesses"]) or "nenhuma"))
66     if state.get("wrong_words"):
67         lines.append("Chutes errados: " + " ".join(state["wrong_words"]))
68     if state["status"] == "AGUARDANDO":
69         lines.append("A partida começa quando o segundo jogador entrar.")
70     elif state["status"] == "PAUSADA":
71         absent = " ".join(p["name"] for p in players if not p["connected"])
72         lines.append(f"Partida pausada. Aguardando reconexão: {absent}.")
73     elif state["status"] == "EM JOGO":
74         current = next(p["name"] for p in players if p["player_id"] == state["turn"])
75         lines.append("Sua vez! Tente uma letra ou chute a palavra." if state["turn"] == player_id
76                     else f"Vez de {current}. Aguarde.")
77     else:
78         lines.append(outcome(state, player_id))
79         lines.append("Digite /nova para jogar novamente.")
80     lines.append("=" * 68)
81     return "\n".join(lines)
82

```

forca/___init___py

3 linhas • SHA-256: c6ab698060287c63...

```

1  """Jogo da forca distribuído por TCP: salas de dois jogadores e servidores primário e reserva com
   estado replicado em RAM."""
2
3  __version__ = "0.2.0"

```

forca/words.txt

20 linhas • SHA-256: 087d5a483d710444...

```
1 PYTHON
2 SOCKET
3 SERVIDOR
4 CLIENTE
5 REDE
6 PROCESSO
7 MEMORIA
8 SISTEMA
9 DOCKER
10 VIRTUAL
11 MENSAGEM
12 PACOTE
13 PROTOCOLO
14 CONEXAO
15 SEMAFORO
16 REPLICA
17 PARTIDA
18 JOGADOR
19 COMPUTADOR
20 ALGORITMO
```

Dockerfile

7 linhas • SHA-256: 0fcda842a842f52f...

```
1 FROM python:3.12.13-slim
2 WORKDIR /app
3 ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
4 COPY forca ./forca
5 COPY servidor.py cliente.py ./
6 EXPOSE 5000 5001
7 CMD ["python", "servidor.py"]
```

compose.yaml

12 linhas • SHA-256: e0b7724338aea854...

```
1 services:
2   servidor:
3     build: .
4     environment:
5       MODO: ${MODO:-primario}
6       PRIMARY_HOST: ${PRIMARY_HOST:-127.0.0.1}
7       REPLICATION_KEY: ${REPLICATION_KEY:-forca-aula}
8     ports:
9       - "5000:5000"
10      - "5001:5001"
11     # Um antigo primário não deve retornar automaticamente como ativo.
12     restart: "no"
```

.env.example

6 linhas • SHA-256: 311aba696fe52584...

```
1 # VM 1: primario. VM 2: reserva.
2 MODO=primario
3 # Na VM 2, informe o IP Bridge da VM 1.
4 PRIMARY_HOST=XXX.XXX.X.XX
5 # Mesmo valor nos dois servidores; rede de aula confiável.
6 REPLICATION_KEY=forca-aula
```

.dockerignore

13 linhas • SHA-256: 03e1c182b3e03776...

```
1 .git
2 .env
3 __pycache__
4 **/__pycache__
5 *.pyc
6 data
7 sessions
8 tmp
9 output
10 .env
11
12 docs/
13 output/
```

pyproject.toml

8 linhas • SHA-256: 17e50d0cd2ca1112...

```
1 [project]
2 name = "forca-distribuida"
3 version = "0.2.0"
4 description = "Jogo da forca distribuído: salas de dois jogadores, sockets TCP e servidores primário e reserva"
5 requires-python = ">=3.12"
6 dependencies = []
7
8
```

scripts/forca.ps1

42 linhas • SHA-256: 294661b9d88f6657...

```

1 param(
2     [Parameter(Position=0)]
3     [ValidateSet('servidor', 'cliente', 'testes')]
4     [string]$Modo = 'servidor',
5     [Parameter(ValueFromRemainingArguments=$true)]
6     [string[]]$Opcoes
7 )
8
9 $ErrorActionPreference = 'Stop'
10 $taskRoot = Split-Path -Parent $PSScriptRoot
11 $pythonExecutable = $env:FORCA_PYTHON
12 $pythonPrefix = @()
13 if (-not $pythonExecutable -and (Test-Path -LiteralPath (Join-Path $taskRoot '.venv/Scripts/pytho
    n.exe'))) {
14     $pythonExecutable = Join-Path $taskRoot '.venv/Scripts/python.exe'
15 }
16 if (-not $pythonExecutable) {
17     $pythonCommand = Get-Command python -ErrorAction SilentlyContinue
18     if ($pythonCommand -and $pythonCommand.Source -notlike '*WindowsApps*') {
19         $pythonExecutable = $pythonCommand.Source
20     }
21 }
22 if (-not $pythonExecutable) {
23     if (Get-Command py -ErrorAction SilentlyContinue) {
24         $pythonExecutable = 'py'
25         $pythonPrefix = @('-3')
26     } else {
27         throw 'Instale Python 3.12+ ou configure FORCA_PYTHON com o caminho do executável.'
28     }
29 }
30
31 Push-Location -LiteralPath $taskRoot
32 try {
33     switch ($Modo) {
34         'servidor' { & $pythonExecutable @pythonPrefix servidor.py @Opcoes }
35         'cliente' { & $pythonExecutable @pythonPrefix cliente.py @Opcoes }
36         'testes' { & $pythonExecutable @pythonPrefix -m unittest discover -s tests -v @Opcoes }
37     }
38     exit $LASTEXITCODE
39 } finally {
40     Pop-Location
41 }
42

```

scripts/gerar_especificacao_pdf.py

151 linhas • SHA-256: 7ac6114f22093163...

```

1  """Gera o PDF da especificação Markdown com ReportLab."""
2  from pathlib import Path
3  import re
4  from xml.sax.saxutils import escape
5
6  from reportlab.lib import colors
7  from reportlab.lib.enums import TA_LEFT
8  from reportlab.lib.pagesizes import A4
9  from reportlab.lib.styles import ParagraphStyle
10 from reportlab.lib.units import mm
11 from reportlab.pdfbase import pdfmetrics
12 from reportlab.pdfbase.ttfonts import TTFont
13 from reportlab.platypus import (
14     SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, Preformatted,
15     KeepTogether,
16 )
17
18 ROOT = Path(__file__).resolve().parents[1]
19 SOURCE = ROOT / 'docs' / 'especificacao-jogo-forca.md'
20 OUTPUT = ROOT / 'output' / 'pdf' / 'especificacao-jogo-forca.pdf'
21 OUTPUT.parent.mkdir(parents=True, exist_ok=True)
22
23 fonts = Path('C:/Windows/Fonts')
24 for name, filename in [('Arial', 'arial.ttf'), ('Arial-Bold', 'arialbd.ttf'),
25                       ('Arial-Italic', 'ariali.ttf'), ('Arial-BoldItalic', 'arialbi.ttf')]:
26     pdfmetrics.registerFont(TTFont(name, str(fonts / filename)))
27 pdfmetrics.registerFontFamily('Arial', normal='Arial', bold='Arial-Bold',
28                               italic='Arial-Italic', boldItalic='Arial-BoldItalic')
29
30 styles = {
31     'title': ParagraphStyle('Title', fontName='Arial-Bold', fontSize=23,
32                             leading=28, textColor=colors.black, spaceAfter=12),
33     'subtitle': ParagraphStyle('Subtitle', fontName='Arial', fontSize=12,
34                                leading=17, spaceAfter=8),
35     'meta': ParagraphStyle('Meta', fontName='Arial', fontSize=9,
36                             leading=13, textColor=colors.HexColor('#555555'), spaceAfter=19),
37     'h1': ParagraphStyle('H1', fontName='Arial-Bold', fontSize=15,
38                           leading=19, spaceBefore=18, spaceAfter=9, keepWithNext=True),
39     'h2': ParagraphStyle('H2', fontName='Arial-Bold', fontSize=11.7,
40                           leading=16, spaceBefore=12, spaceAfter=7, keepWithNext=True),
41     'body': ParagraphStyle('Body', fontName='Arial', fontSize=10.2,
42                             leading=14.3, spaceAfter=8, allowWidows=0, allowOrphans=0),
43     'list': ParagraphStyle('List', fontName='Arial', fontSize=10.2,
44                             leading=14.3, spaceAfter=6, leftIndent=15, firstLineIndent=0,
45                             bulletIndent=0, allowWidows=0, allowOrphans=0),
46     'cell': ParagraphStyle('Cell', fontName='Arial', fontSize=9.3,
47                             leading=12.4, spaceAfter=0),
48     'head': ParagraphStyle('Head', fontName='Arial-Bold', fontSize=9.3,
49                             leading=12.4, textColor=colors.white),
50     'code': ParagraphStyle('Code', fontName='Courier', fontSize=8.8,
51                             leading=12, spaceBefore=5, spaceAfter=10),
52 }
53
54 def markup(text):
55     out = escape(text)
56     out = re.sub(r'\*\*(.+?)\*\*', r'<b>\1</b>', out)
57     out = re.sub(r'(https?:\/\/[^\s]+)',
58                 lambda m: '<link href="' + m[1] + '" color="#244867">' + m[1] + '</link>', out)
59     return out
60
61 WIDTH = A4[0] - 40*mm
62
63 def make_table(lines):
64     raw = [[cell.strip() for cell in line.strip('|').split('|')] for line in lines]
65     raw = [r for r in raw if not all(re.fullmatch(r'[:\s- ]+', c) for c in r)]
66     n = len(raw[0])
67     if n == 2:
68         first = max(len(row[0]) for row in raw)
69         ratio = .38 if first > 25 else .30
70         widths = [WIDTH*ratio, WIDTH*(1-ratio)]

```

```

71     else:
72         widths = [WIDTH*.10, WIDTH*.37, WIDTH*.53] if raw[0][0] == 'ID' else [WIDTH*.25, WIDTH*.2
9, WIDTH*.46]
73     rows = [[Paragraph(markup(c), styles['head' if i == 0 else 'cell'])
74               for c in row] for i, row in enumerate(raw)]
75     table = Table(rows, colWidths=widths, repeatRows=1, hAlign='LEFT')
76     commands = [('BACKGROUND', (0,0), (-1,0), colors.HexColor('#244867')),
77                 ('GRID', (0,0), (-1,-1), .45, colors.HexColor('#D9D9D9')),
78                 ('VALIGN', (0,0), (-1,-1), 'MIDDLE'),
79                 ('LEFTPADDING', (0,0), (-1,-1), 8),
80                 ('RIGHTPADDING', (0,0), (-1,-1), 8),
81                 ('TOPPADDING', (0,0), (-1,-1), 6),
82                 ('BOTTOMPADDING', (0,0), (-1,-1), 6)]
83     for i in range(1, len(rows)):
84         commands.append(('BACKGROUND', (0,i), (-1,i),
85                         colors.HexColor('#F2F5F7') if i % 2 == 0 else colors.white))
86     table.setStyle(TableStyle(commands))
87     return table
88
89 def page_decor(canvas, doc):
90     canvas.saveState()
91     canvas.setFont('Arial', 8)
92     canvas.setFillColor(colors.HexColor('#555555'))
93     if doc.page > 1:
94         canvas.drawString(20*mm, A4[1]-13*mm, 'Jogo da força distribuído | Especificação técnica'
95     )
96     canvas.drawString(20*mm, 12*mm, 'Sistemas Distribuídos | Versão 1.0')
97     canvas.drawRightString(A4[0]-20*mm, 12*mm, str(doc.page))
98     canvas.restoreState()
99
100 class SpecDoc(SimpleDocTemplate):
101     def afterFlowable(self, flowable):
102         if isinstance(flowable, Paragraph) and flowable.style.name == 'H1':
103             title = flowable.getPlainText()
104             key = 'section-' + title.split(' ')[0]
105             self.canv.bookmarkPage(key)
106             self.canv.addOutlineEntry(title, key, 0, False)
107
108 lines = SOURCE.read_text(encoding='utf-8').splitlines()
109 story = []
110 i = 0
111 while i < len(lines):
112     line = lines[i].strip()
113     if not line:
114         i += 1
115         continue
116     if line.startswith('```'):
117         chunk = []
118         i += 1
119         while i < len(lines) and not lines[i].startswith('```'):
120             chunk.append(lines[i])
121             i += 1
122         story.append(Preformatted('\n'.join(chunk), styles['code']))
123     elif line.startswith('|'):
124         block = []
125         while i < len(lines) and lines[i].startswith('|'):
126             block.append(lines[i])
127             i += 1
128         story.extend([make_table(block), Spacer(1, 9)])
129         continue
130     elif line.startswith('### '):
131         story.append(Paragraph(markup(line[4:]), styles['h2']))
132     elif line.startswith('## '):
133         story.append(Paragraph(markup(line[3:]), styles['h1']))
134     elif line.startswith('# '):
135         story.append(Paragraph(markup(line[2:]), styles['title']))
136     elif line.startswith('- '):
137         story.append(Paragraph(markup(line[2:]), styles['list'], bulletText='•'))
138     elif re.match(r'^\d+\.', line):
139         num, content = line.split('. ', 1)
140         story.append(Paragraph(markup(content), styles['list'], bulletText=num+'.'))
141     else:
142         style = 'subtitle' if i == 2 else 'meta' if i == 4 else 'body'
143         story.append(Paragraph(markup(line), styles[style]))

```

```

143     i += 1
144
145 doc = SpecDoc(str(OUTPUT), pagesize=A4, rightMargin=20*mm,
146               leftMargin=20*mm, topMargin=22*mm, bottomMargin=21*mm,
147               title='Especificação do jogo da força distribuído',
148               author='Projeto de Sistemas Distribuídos',
149               subject='Python, sockets, duas máquinas, VMs e Docker')
150 doc.build(story, onFirstPage=page_decor, onLaterPages=page_decor)
151 print(OUTPUT)

```

tests/test_game.py

141 linhas • SHA-256: f92ce61a97ef89b4...

```

1  """Regras de uma partida, sem rede."""
2  import unittest
3
4  from forca.game import GameError, Room, load_words, normalize
5
6
7  def started(word="SOCKET"):
8      room = Room("sala-1", players=["ana", "bruno"])
9      room.start(word)
10     return room
11
12
13 class LetterTests(unittest.TestCase):
14     def test_primeiro_a_entrar_comeca(self):
15         self.assertEqual(started().turn, "ana")
16
17     def test_fora_da_vez_nao_altera_a_sala(self):
18         room = started()
19         with self.assertRaises(GameError) as error:
20             room.guess("bruno", "S", room.version)
21         self.assertEqual(error.exception.code, "FORA_DA_VEZ")
22         self.assertEqual((room.turn, room.guesses), ("ana", []))
23
24     def test_versao_desatualizada(self):
25         room = started()
26         with self.assertRaises(GameError) as error:
27             room.guess("ana", "S", room.version - 1)
28         self.assertEqual(error.exception.code, "ESTADO_DESATUALIZADO")
29
30     def test_letra_invalida_e_repetida_nao_passam_a_vez(self):
31         room = started()
32         for letter in ("", "AB", "1", "-", None):
33             with self.assertRaises(GameError) as error:
34                 room.guess("ana", letter, room.version)
35             self.assertEqual(error.exception.code, "LETRA_INVALIDA")
36         room.guess("ana", "s", room.version)
37         with self.assertRaises(GameError) as error:
38             room.guess("bruno", "S", room.version)
39         self.assertEqual(error.exception.code, "LETRA_REPETIDA")
40         self.assertEqual(room.turn, "bruno")
41
42     def test_letra_com_acento_vale_como_letra_sem_acento(self):
43         room = started("CONEXAO")
44         self.assertTrue(room.guess("ana", "ç", room.version))
45         self.assertTrue(room.guess("bruno", "Ã", room.version))
46         self.assertEqual(room.guesses, ["C", "A"])
47
48     def test_erro_conta_so_para_o_autor_e_passa_a_vez(self):
49         room = started()
50         self.assertFalse(room.guess("ana", "Z", room.version))
51         self.assertEqual(room.errors, {"ana": 1, "bruno": 0})
52         self.assertEqual(room.turn, "bruno")
53
54     def test_completar_a_palavra_vence(self):
55         room = started("REDE")
56         for player, letter in (("ana", "R"), ("bruno", "E"), ("ana", "D")):
57             room.guess(player, letter, room.version)

```

```

58         self.assertEqual((room.status, room.winner, room.reason), ("ENCERRADA", "ana", "PALAVRA_C
OMPLETA"))
59
60     def test_seis_erros_da_vitoria_ao_adversario(self):
61         room = started()
62         for ana, bruno in zip("ABDFGH", "IJLMNP"):
63             room.guess("ana", ana, room.version)
64             if room.status == "EM_JOGO":
65                 room.guess("bruno", bruno, room.version)
66         self.assertEqual(room.errors["ana"], 6)
67         self.assertEqual((room.status, room.winner, room.reason), ("ENCERRADA", "bruno", "SEIS_ER
ROS"))
68
69
70 class WordGuessTests(unittest.TestCase):
71     def test_chute_certo_vence_com_palavra_completa(self):
72         room = started()
73         self.assertTrue(room.guess_word("ana", " socket ", room.version))
74         self.assertEqual((room.status, room.winner, room.reason), ("ENCERRADA", "ana", "PALAVRA_C
OMPLETA"))
75         self.assertEqual(room.errors["ana"], 0)
76
77     def test_chute_errado_custa_um_erro_e_passa_a_vez(self):
78         room = started()
79         version = room.version
80         self.assertFalse(room.guess_word("ana", "SERVIDOR", version))
81         self.assertEqual(room.errors, {"ana": 1, "bruno": 0})
82         self.assertEqual((room.turn, room.status, room.version), ("bruno", "EM_JOGO", version + 1
))
83         self.assertEqual(room.wrong_words, ["SERVIDOR"])
84
85     def test_chute_repetido_ou_invalido_nao_penaliza(self):
86         room = started()
87         room.guess_word("ana", "REDE", room.version)
88         with self.assertRaises(GameError) as error:
89             room.guess_word("bruno", "rede", room.version)
90         self.assertEqual(error.exception.code, "PALAVRA_REPETIDA")
91         for word in ("", " ", "DUAS PALAVRAS", "GUARDA-CHUVA", "X" * 31, 42):
92             with self.assertRaises(GameError) as error:
93                 room.guess_word("bruno", word, room.version)
94         self.assertEqual(error.exception.code, "PALAVRA_INVALIDA")
95         self.assertEqual((room.errors["bruno"], room.turn), (0, "bruno"))
96
97     def test_chute_com_acento_acerta_palavra_sem_acento(self):
98         room = started("CONEXAO")
99         self.assertTrue(room.guess_word("ana", "conexão", room.version))
100         self.assertEqual(room.reason, "PALAVRA_COMPLETA")
101
102     def test_chute_fora_da_vez(self):
103         room = started()
104         with self.assertRaises(GameError) as error:
105             room.guess_word("bruno", "SOCKET", room.version)
106         self.assertEqual(error.exception.code, "FORA_DA_VEZ")
107
108     def test_sexto_erro_em_chute_encerra(self):
109         room = started()
110         room.errors["ana"] = 5
111         room.guess_word("ana", "REDE", room.version)
112         self.assertEqual((room.status, room.winner, room.reason), ("ENCERRADA", "bruno", "SEIS_ER
ROS"))
113
114     def test_estado_publico_mostra_chutes_e_esconde_palavra(self):
115         from forca.game import Player
116         room = started()
117         room.guess_word("ana", "REDE", room.version)
118         players = {p: Player(p, p.title(), p) for p in room.players}
119         public = room.public(players, {"ana", "bruno"})
120         self.assertEqual(public["wrong_words"], ["REDE"])
121         self.assertEqual(public["masked_word"], "_____")
122
123
124 class WordListTests(unittest.TestCase):
125     def test_normaliza_acentos_e_maiusculas(self):
126         self.assertEqual(normalize("Coração"), "CORACAO")

```

```

127         self.assertEqual(load_words([" socket ", "", "coração", "Conexão"]), ["SOCKET", "CORACAO",
128         ", "CONEXAO"])
129
130     def test_recusa_palavras_que_nao_sao_so_letras(self):
131         for lines in (["BOA NOITE"], ["GUARDA-CHUVA"], ["SALA1"], ["X" * 31], [], [",", " "]):
132             with self.assertRaises(ValueError):
133                 load_words(lines)
134
135     def test_lista_do_projeto_e_valida(self):
136         from pathlib import Path
137         path = Path(__file__).resolve().parents[1] / "forca" / "words.txt"
138         self.assertTrue(load_words(path.read_text(encoding="utf-8").splitlines()))
139
140 if __name__ == "__main__":
141     unittest.main()

```

tests/test_lobby.py

188 linhas • SHA-256: 97816530f322eaf3...

```

1  """Salas, sessões, nomes únicos e reenvios."""
2  import unittest
3  import uuid
4
5  from forca.lobby import apply, new_state, player_id, public_state
6
7  WORDS = ["SOCKET"]
8
9
10 def token(name):
11     return (name + "-token-").ljust(40, "x")
12
13
14 class Lobby:
15     """Estado de um servidor sem rede; todos os jogadores criados estão online."""
16
17     def __init__(self):
18         self.state = new_state()
19         self.online = set()
20         self.idle = set()
21
22     def send(self, who, kind, request_id=None, **fields):
23         self.online.add(player_id(token(who)))
24         command = {"type": kind, "token": token(who),
25                   "request_id": request_id or str(uuid.uuid4()), **fields}
26         return apply(self.state, command, self.online, WORDS, self.idle)
27
28     def enter(self, who, display=None):
29         return self.send(who, "ENTRAR", name=who if display is None else display)
30
31     def view(self, who):
32         return public_state(self.state, player_id(token(who)), self.online)
33
34
35 class RoomTests(unittest.TestCase):
36     def test_salas_sao_formadas_em_ordem(self):
37         lobby = Lobby()
38         messages = [lobby.enter(n)["message"] for n in ("Ana", "Bruno", "Caio", "Dani", "Edu")]
39         self.assertEqual(messages, ["Você está na sala-1.", "Você está na sala-1.", "Você está na
40         sala-2.", "Você está na sala-2.", "Você está na sala-3."])
41         self.assertEqual(lobby.view("Ana")["status"], "EM_JOGO")
42         self.assertEqual(lobby.view("Edu")["status"], "AGUARDANDO")
43         self.assertEqual(len(lobby.view("Caio")["players"]), 2)
44
45     def test_entrar_de_novo_mantem_a_sala(self):
46         lobby = Lobby()
47         lobby.enter("Ana")
48         lobby.enter("Bruno")
49         self.assertEqual(lobby.enter("Ana")["message"], "Você já está em uma partida na sala-1. U

```

```

se /sair para desistir.")
50     self.assertEqual(lobby.view("Ana")["room_version"], lobby.view("Bruno")["room_version"])
51
52     def test_quem_espera_sozinho_continua_na_sala(self):
53         lobby = Lobby()
54         lobby.enter("Ana")
55         self.assertEqual(lobby.enter("Ana")["message"], "Você continua aguardando na sala-1.")
56
57     def test_dois_esperando_em_salas_separadas_se_juntam(self):
58         lobby = Lobby()
59         lobby.enter("Ana")
60         lobby.online.discard(player_id(token("Ana"))) # Ana cai; Bruno não a encontra e abre out
ra sala.
61         self.assertEqual(lobby.enter("Bruno")["message"], "Você está na sala-2.")
62         self.assertEqual(lobby.enter("Ana")["message"], "Você está na sala-2.") # Ana volta e se
junta.
63         view = lobby.view("Ana")
64         self.assertEqual((view["room_id"], view["status"]), ("sala-2", "EM_JOGO"))
65         self.assertEqual(lobby.state["world"]["rooms"]["sala-1"]["status"], "CANCELADA")
66
67     def test_jogador_ausente_pausa_a_partida(self):
68         lobby = Lobby()
69         lobby.enter("Ana")
70         lobby.enter("Bruno")
71         lobby.online.discard(player_id(token("Bruno")))
72         self.assertEqual(lobby.view("Ana")["status"], "PAUSADA")
73         state = lobby.view("Ana")
74         result = lobby.send("Ana", "JOGAR", letter="S", version=state["room_version"])
75         self.assertEqual(result["code"], "PAUSADA")
76
77     def test_sair_na_espera_cancela_e_na_partida_da_vitoria(self):
78         lobby = Lobby()
79         lobby.enter("Ana")
80         lobby.send("Ana", "SAIR")
81         lobby.enter("Bruno")
82         lobby.enter("Caio")
83         lobby.send("Bruno", "SAIR")
84         self.assertIsNone(lobby.view("Ana"))
85         view = lobby.view("Caio")
86         self.assertEqual((view["status"], view["reason"]), ("ENCERRADA", "DESISTENCIA"))
87         self.assertEqual(view["winner"], player_id(token("Caio")))
88         self.assertEqual(view["masked_word"], "SOCKET")
89
90     def test_chutar_pelo_lobby(self):
91         lobby = Lobby()
92         lobby.enter("Ana")
93         lobby.enter("Bruno")
94         version = lobby.view("Ana")["room_version"]
95         wrong = lobby.send("Ana", "CHUTAR", word="REDE", version=version)
96         self.assertTrue(wrong["ok"])
97         self.assertIn("não coincide", wrong["message"])
98         right = lobby.send("Bruno", "CHUTAR", word="socket", version=version + 1)
99         self.assertEqual(right["message"], "Acertou a palavra!")
100         view = lobby.view("Ana")
101         self.assertEqual((view["status"], view["reason"]), ("ENCERRADA", "PALAVRA_COMPLETA"))
102         self.assertEqual([p["errors"] for p in view["players"]], [1, 0])
103
104 class UniqueNameTests(unittest.TestCase):
105     def test_nome_repetido_e_recusado_em_qualquer_sala(self):
106         lobby = Lobby()
107         lobby.enter("ana1", "Ana")
108         self.assertEqual(lobby.enter("ana2", "ana")["code"], "NOME_EM_USO")
109         lobby.enter("Bruno") # Sala 1 começa; a regra continua valendo para a sala 2.
110         self.assertEqual(lobby.enter("ana3", "ANA ")["code"], "NOME_EM_USO")
111         self.assertIsNone(lobby.view("ana2"))
112         self.assertEqual(len(lobby.view("ana1")["players"]), 2)
113
114     def test_mesmo_jogador_pode_reentrar_com_seu_nome(self):
115         lobby = Lobby()
116         lobby.enter("ana1", "Ana")
117         self.assertTrue(lobby.enter("ana1", "Ana")["ok"])
118
119     def test_sair_libera_o_nome(self):

```

```

121     lobby = Lobby()
122     lobby.enter("ana1", "Ana")
123     lobby.send("ana1", "SAIR")
124     self.assertTrue(lobby.enter("ana2", "Ana")["ok"])
125     self.assertEqual(lobby.enter("ana1", "Ana")["code"], "NOME_EM_USO")
126
127     def test_nome_de_quem_sumiu_fora_de_partida_e_liberado(self):
128         lobby = Lobby()
129         lobby.enter("ana1", "Ana")
130         lobby.online.discard(player_id(token("ana1")))
131         lobby.idle.add(player_id(token("ana1")))
132         self.assertEqual(lobby.enter("ana2", "Ana")["message"], "Você está na sala-2.")
133         self.assertIsNone(lobby.view("ana1"))
134         self.assertEqual(lobby.state["world"]["rooms"]["sala-1"]["status"], "CANCELADA")
135         self.assertEqual(lobby.enter("ana1", "Ana")["code"], "NOME_EM_USO")
136
137     def test_nome_de_quem_esta_em_partida_nao_e_liberado(self):
138         lobby = Lobby()
139         lobby.enter("ana1", "Ana")
140         lobby.enter("Bruno")
141         lobby.idle.add(player_id(token("ana1")))
142         self.assertEqual(lobby.enter("ana2", "Ana")["code"], "NOME_EM_USO")
143
144     def test_nome_invalido(self):
145         lobby = Lobby()
146         for name in ("", " ", "x" * 25, "Ana\n"):
147             self.assertEqual(lobby.enter("p", name)["code"], "NOME_INVALIDO")
148
149
150 class ResendTests(unittest.TestCase):
151     def test_reenvio_devolve_o_mesmo_resultado_sem_repetir(self):
152         lobby = Lobby()
153         lobby.enter("Ana")
154         lobby.enter("Bruno")
155         version = lobby.view("Ana")["room_version"]
156         request_id = str(uuid.uuid4())
157         first = lobby.send("Ana", "JOGAR", request_id, letter="Z", version=version)
158         again = lobby.send("Ana", "JOGAR", request_id, letter="Z", version=version)
159         self.assertEqual(first, again)
160         self.assertEqual(lobby.view("Ana")["players"][0]["errors"], 1)
161         self.assertEqual(lobby.view("Ana")["room_version"], version + 1)
162
163     def test_mesmo_identificador_com_outro_conteudo(self):
164         lobby = Lobby()
165         request_id = str(uuid.uuid4())
166         lobby.send("Ana", "ENTRAR", request_id, name="Ana")
167         with self.assertRaises(ValueError):
168             lobby.send("Ana", "ENTRAR", request_id, name="Outra")
169
170     def test_token_e_identificador_invalidos(self):
171         with self.assertRaises(ValueError):
172             player_id("curto")
173         for request_id in ("nao-e-uuid", 5, ["lista"]):
174             with self.assertRaises(ValueError):
175                 Lobby().send("Ana", "ENTRAR", request_id, name="Ana")
176
177     def test_recibos_nao_crescem_com_comandos_rejeitados(self):
178         lobby = Lobby()
179         lobby.enter("Ana")
180         for i in range(200):
181             lobby.send("Ana", "JOGAR", letter="1", version=0)
182             lobby.send(f"intruso{i}", "JOGAR", letter="A", version=0) # Nunca entrou: nada a gua
183         self.assertEqual(len(lobby.state["receipts"]), 1)
184         self.assertEqual(lobby.state["revision"], 401)
185
186
187 if __name__ == "__main__":
188     unittest.main()

```

tests/test_cliente.py

95 linhas • SHA-256: 32e913cfdc1ae6a8...

```

1  """Enquadramento das mensagens, tela e comandos do cliente."""
2  import io
3  from pathlib import Path
4  import tempfile
5  import unittest
6
7  from cliente import lock, parse
8  from forca.ui import HELP, outcome, render
9  from forca.wire import MAX_BYTES, receive, send
10
11  STATE = {"room_id": "sala-1", "room_version": 3, "status": "EM_JOGO",
12          "players": [{"player_id": "a", "name": "Ana", "errors": 1, "connected": True},
13                    {"player_id": "b", "name": "Bruno", "errors": 0, "connected": True}],
14          "masked_word": "S____T", "guesses": ["S", "T", "Z"], "wrong_words": ["REDE"],
15          "turn": "a", "winner": None, "reason": None, "max_errors": 6}
16
17
18  class WireTests(unittest.TestCase):
19      def test_ida_e_volta(self):
20          stream = io.BytesIO()
21          send(stream, {"type": "PING", "texto": "ação"})
22          send(stream, {"type": "ESTADO"})
23          stream.seek(0)
24          self.assertEqual(receive(stream), {"type": "PING", "texto": "ação"})
25          self.assertEqual(receive(stream), {"type": "ESTADO"})
26          with self.assertRaises(ConnectionError):
27              receive(stream)
28
29      def test_mensagens_invalidas(self):
30          for data in (b'{"sem": "quebra"}', b"[1, 2]\n", b"nao e json\n", b"x" * (MAX_BYTES + 2) +
31                    b"\n"):
32              with self.assertRaises(ValueError):
33                  receive(io.BytesIO(data))
34
35  class ScreenTests(unittest.TestCase):
36      def test_estado_nao_lista_comandos(self):
37          screen = render(STATE, "a")
38          for expected in ("sala-1", "Ana (você)", "Bruno", "S _ _ _ T", "S, T, Z", "Chutes errad
39                           os: REDE",
40                           "Erros: 1/6", "Sua vez!"):
41              self.assertIn(expected, screen)
42          for command in ("/ajuda", "/help", "/estado", "/sair"):
43              self.assertNotIn(command, screen)
44
45      def test_sem_sala(self):
46          self.assertIn("não está em nenhuma sala", render(None, "a"))
47
48      def test_resultado_do_ponto_de_vista_de_cada_jogador(self):
49          def ended(winner, reason):
50              return {**STATE, "status": "ENCERRADA", "turn": None, "winner": winner, "reason": rea
51                      son}
52          cases = [{"a", "PALAVRA_COMPLETA", "a", "Você venceu! Você completou a palavra."},
53                  {"a", "PALAVRA_COMPLETA", "b", "Você perdeu. Ana completou a palavra."},
54                  {"b", "SEIS_ERROS", "b", "Você venceu! Ana chegou a seis erros."},
55                  {"b", "SEIS_ERROS", "a", "Você perdeu. Você chegou a seis erros."},
56                  {"a", "DESISTENCIA", "a", "Você venceu! Bruno desistiu."}]
57          for winner, reason, viewer, expected in cases:
58              self.assertEqual(outcome(ended(winner, reason), viewer), expected)
59              self.assertNotIn(reason, render(ended(winner, reason), viewer))
60          self.assertEqual(outcome({**STATE, "winner": None, "reason": "DESISTENCIA"}, "a"), "Parti
61                           da cancelada.")
62
63      def test_ajuda_lista_apenas_comandos(self):
64          for command in ("/chute", "/estado", "/nova", "/sair", "/ajuda", "/help"):
65              self.assertIn(command, HELP)
66              self.assertNotIn("sala-1", HELP)
67              self.assertNotIn("+-----+", HELP)

```

```

67 class ParseTests(unittest.TestCase):
68     def test_comandos(self):
69         self.assertEqual(parse("a", STATE), {"type": "JOGAR", "letter": "a", "version": 3})
70         self.assertEqual(parse("ç", STATE), {"type": "JOGAR", "letter": "ç", "version": 3})
71         self.assertEqual(parse("/chute socket", STATE), {"type": "CHUTAR", "word": "socket", "ver
sion": 3})
72         self.assertEqual(parse("/CHUTE rede ", STATE), {"type": "CHUTAR", "word": "rede", "versi
on": 3})
73         self.assertEqual(parse("/nova", None), {"type": "ENTRAR"})
74         self.assertEqual(parse("/sair", STATE), {"type": "SAIR"})
75
76     def test_avisos_locais(self):
77         for action in ("/chute", "/chute ", "/xyz", "ab"):
78             self.assertIsInstance(parse(action, STATE), str)
79
80
81 class SessionLockTests(unittest.TestCase):
82     def test_segundo_cliente_com_a_mesma_sessao_e_recusado(self):
83         with tempfile.TemporaryDirectory() as folder:
84             path = Path(folder) / "ana.json"
85             first = lock(path)
86             self.assertIsNotNone(first)
87             self.assertIsNone(lock(path))
88             first.close()
89             second = lock(path)
90             self.assertIsNotNone(second)
91             second.close()
92
93
94 if __name__ == "__main__":
95     unittest.main()

```

tests/test_servidor.py

201 linhas • SHA-256: ce0091cc489ec669...

```

1  """Primário e reserva em processos reais, com TCP, replicação e queda abrupta."""
2  import os
3  from pathlib import Path
4  import socket
5  import subprocess
6  import sys
7  import threading
8  import time
9  import unittest
10 import uuid
11
12 from forca.lobby import new_state
13 from forca.wire import receive, send
14
15 ROOT = Path(__file__).resolve().parents[1]
16
17
18 def free_port():
19     with socket.socket() as probe:
20         probe.bind(("127.0.0.1", 0))
21         return probe.getsockname()[1]
22
23
24 def exchange(port, command):
25     with socket.create_connection(("127.0.0.1", port), timeout=2) as connection:
26         connection.settimeout(5)
27         with connection.makefile("rwb") as stream:
28             send(stream, command)
29             return receive(stream)
30
31
32 def token(name):
33     return (name + "-token-").ljust(40, "x")
34
35

```

```

36 def command(who, kind, request_id=None, **fields):
37     return {"type": kind, "token": token(who), "request_id": request_id or str(uuid.uuid4()), **fields}
38
39
40 def wait_for(check, seconds=15):
41     limit = time.monotonic() + seconds
42     while time.monotonic() < limit:
43         try:
44             result = check()
45             if result:
46                 return result
47         except OSError:
48             pass
49         time.sleep(0.2)
50     raise AssertionError("Condição não atingida a tempo.")
51
52
53 class Processes(unittest.TestCase):
54     """Sobe servidor.py em subprocessos e encerra todos ao fim de cada teste."""
55
56     def setUp(self):
57         self.processes = []
58
59     def start(self, *arguments):
60         environment = {**os.environ, "PYTHONUNBUFFERED": "1", "PYTHONIOENCODING": "utf-8"}
61         process = subprocess.Popen([sys.executable, "servidor.py", *arguments], cwd=ROOT, env=environment,
62                                     stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
63         self.processes.append(process)
64         return process
65
66     def stop(self, process):
67         process.kill()
68         output = process.communicate(timeout=10)[0]
69         return output.decode("utf-8", "replace")
70
71     def tearDown(self):
72         for process in self.processes:
73             if process.poll() is None:
74                 self.stop(process)
75             else:
76                 process.communicate()
77
78
79 class ReplicationTests(Processes):
80     def setUp(self):
81         super().setUp()
82         self.game, self.sync, self.backup_game = free_port(), free_port(), free_port()
83         self.primary = self.start("--port", str(self.game), "--sync-port", str(self.sync))
84         wait_for(lambda: exchange(self.game, {"type": "PING"}))
85         # Sem reserva sincronizado, nenhuma alteração é aceita.
86         self.assertTrue(exchange(self.game, command("Ana", "ENTRAR", name="Ana")).get("retry"))
87         self.backup = self.start("--modo", "reserva", "--port", str(self.backup_game),
88                                   "--sync-port", str(self.sync))
89         # ESTADO recebe retry até o reserva sincronizar.
90         wait_for(lambda: "retry" not in exchange(self.game, {"type": "ESTADO", "token": token("x")
91     )))
92
93     def test_reserva_assume_com_a_partida_intacta(self):
94         exchange(self.game, command("Ana", "ENTRAR", name="Ana"))
95         state = exchange(self.game, command("Bruno", "ENTRAR", name="Bruno"))["state"]
96         request_id = str(uuid.uuid4())
97         first = exchange(self.game, command("Ana", "JOGAR", request_id, letter="Z", version=state
98     ["room_version"]))
99         self.assertTrue(first["ok"])
100         exchange(self.game, command("Bruno", "ESTADO"))
101         before = exchange(self.game, command("Ana", "ESTADO"))["state"]
102
103         self.stop(self.primary)
104         wait_for(lambda: exchange(self.backup_game, {"type": "PING"}))
105         exchange(self.backup_game, command("Bruno", "ESTADO"))
106         after = exchange(self.backup_game, command("Ana", "ESTADO"))
107         self.assertEqual(after["role"], "RESERVA_PROMOVIDO")

```

```

106         self.assertEqual(after["state"], before)
107
108         # O reenvio da jogada já confirmada não é aplicado de novo no reserva.
109         again = exchange(self.backup_game, command("Ana", "JOGAR", request_id, letter="Z",
110                                                    version=state["room_version"]))
111         self.assertEqual(again["message"], first["message"])
112         self.assertEqual(again["state"]["room_version"], before["room_version"])
113
114         guess = exchange(self.backup_game, command("Bruno", "CHUTAR", word="XYZ", version=before[
115 "room_version"]))
116         self.assertTrue(guess["ok"])
117         self.assertEqual(guess["state"]["wrong_words"], ["XYZ"])
118
119     def test_nome_unico_vale_entre_os_servidores(self):
120         exchange(self.game, command("ana1", "ENTRAR", name="Ana"))
121         self.stop(self.primary)
122         wait_for(lambda: exchange(self.backup_game, {"type": "PING"}))
123         refused = exchange(self.backup_game, command("ana2", "ENTRAR", name="ANA"))
124         self.assertEqual(refused["code"], "NOME_EM_USO")
125
126     def test_primario_pausa_ao_perder_o_reserva(self):
127         self.stop(self.backup)
128         wait_for(lambda: "retry" in exchange(self.game, command("Ana", "ENTRAR", name="Ana")))
129         invalid = socket.create_connection(("127.0.0.1", self.game), timeout=2)
130         with invalid, invalid.makefile("rwb") as stream:
131             stream.write(b"isto nao e json\n")
132             stream.flush()
133             self.assertFalse(receive(stream)["ok"])
134         log = self.stop(self.primary)
135         self.assertIn("primário pausado", log.lower()) # Por ACK ou heartbeat.
136         self.assertNotIn("Traceback", log)
137
138     def test_comando_malformatado_recebe_resposta_sem_traceback(self):
139         response = exchange(self.game, {"type": "JOGAR", "token": token("Ana"), "request_id": 5})
140         self.assertFalse(response["ok"])
141         self.assertIn("Comando inválido", response["message"])
142         self.assertNotIn("Traceback", self.stop(self.primary))
143
144
145 class InitialSyncTests(Processes):
146     def setUp(self):
147         super().setUp()
148         self.game, self.sync, self.backup_game = free_port(), free_port(), free_port()
149
150     def ready(self):
151         return "retry" not in exchange(self.game, {"type": "ESTADO", "token": token("x")})
152
153     def test_chave_com_acento_e_reserva_que_some_nao_travam_o_primario(self):
154         primary = self.start("--port", str(self.game), "--sync-port", str(self.sync))
155         wait_for(lambda: exchange(self.game, {"type": "PING"}))
156         with socket.create_connection(("127.0.0.1", self.sync), timeout=2) as intruder:
157             with intruder.makefile("rwb") as stream:
158                 send(stream, {"key": "ç"})
159                 self.assertEqual(stream.readline(), b"") # Recusado: conexão fechada.
160         with socket.create_connection(("127.0.0.1", self.sync), timeout=2) as vanishing:
161             with vanishing.makefile("rwb") as stream:
162                 send(stream, {"key": "forca-aula"})
163                 self.assertIn("state", receive(stream)) # Recebe a cópia e some sem ACK.
164         self.start("--modo", "reserva", "--port", str(self.backup_game), "--sync-port", str(self.
sync))
165         wait_for(self.ready)
166         log = self.stop(primary)
167         self.assertIn("Sincronização inicial com o reserva falhou", log)
168         self.assertNotIn("pausado", log.lower())
169         self.assertNotIn("Traceback", log)
170
171     def test_reserva_so_assume_apos_a_segunda_mensagem(self):
172         listener = socket.create_server(("127.0.0.1", self.sync))
173         self.addCleanup(listener.close)
174         listener.settimeout(15)
175
176     def fake_primary(heartbeat):
177         connection, _ = listener.accept()

```

```

178         with connection, connection.makefile("rwb") as stream:
179             receive(stream)
180             send(stream, {"state": new_state()})
181             receive(stream)
182             if heartbeat:
183                 send(stream, {"heartbeat": True})
184                 receive(stream)
185
186         backup = self.start("--modo", "reserva", "--port", str(self.backup_game),
187                             "--sync-port", str(self.sync))
188         fake_primary(heartbeat=False) # Só a cópia inicial: o ACK pode não ter chegado ao primár
189     io.
189         time.sleep(1.5)
190         with self.assertRaises(SError):
191             exchange(self.backup_game, {"type": "PING"})
192         thread = threading.Thread(target=fake_primary, args=(True,))
193         thread.start() # 0 reserva tenta de novo; agora recebe a segunda mensagem.
194         thread.join(15)
195         self.assertEqual(wait_for(lambda: exchange(self.backup_game, {"type": "PING"}))["role"],
196                           "RESERVA_PROMOVIDO")
197         self.assertNotIn("Traceback", self.stop(backup))
198
199
200 if __name__ == "__main__":
201     unittest.main()

```

.gitignore

222 linhas • SHA-256: bcd4062ef0912639...

```

1  # Byte-compiled / optimized / DLL files
2  __pycache__/
3  *.py[cod]
4  *$py.class
5
6  # C extensions
7  *.so
8
9  # Distribution / packaging
10 .Python
11 build/
12 develop-eggs/
13 dist/
14 downloads/
15 eggs/
16 .eggs/
17 lib/
18 lib64/
19 parts/
20 sdist/
21 var/
22 wheels/
23 share/python-wheels/
24 *.egg-info/
25 .installed.cfg
26 *.egg
27 MANIFEST
28
29 # PyInstaller
30 #   Usually these files are written by a python script from a template
31 #   before PyInstaller builds the exe, so as to inject date/other infos into it.
32 *.manifest
33 *.spec
34
35 # Installer logs
36 pip-log.txt
37 pip-delete-this-directory.txt
38
39 # Unit test / coverage reports
40 htmlcov/
41 .tox/

```

```
42 .nox/
43 .coverage
44 .coverage.*
45 .cache
46 nosetests.xml
47 coverage.xml
48 *.cover
49 *.py.cover
50 .hypothesis/
51 .pytest_cache/
52 cover/
53
54 # Translations
55 *.mo
56 *.pot
57
58 # Django stuff:
59 *.log
60 local_settings.py
61 db.sqlite3
62 db.sqlite3-journal
63
64 # Flask stuff:
65 instance/
66 .webassets-cache
67
68 # Scrappy stuff:
69 .scrappy
70
71 # Sphinx documentation
72 docs/_build/
73
74 # PyBuilder
75 .pybuilder/
76 target/
77
78 # Jupyter Notebook
79 .ipynb_checkpoints
80
81 # IPython
82 profile_default/
83 ipython_config.py
84
85 # pyenv
86 # For a library or package, you might want to ignore these files since the code is
87 # intended to run in multiple environments; otherwise, check them in:
88 # .python-version
89
90 # pipenv
91 # According to pypa/pipenv#598, it is recommended to include Pipfile.lock in version control.
92 # However, in case of collaboration, if having platform-specific dependencies or dependencies
93 # having no cross-platform support, pipenv may install dependencies that don't work, or not
94 # install all needed dependencies.
95 # Pipfile.lock
96
97 # UV
98 # Similar to Pipfile.lock, it is generally recommended to include uv.lock in version control.
99 # This is especially recommended for binary packages to ensure reproducibility, and is more
100 # commonly ignored for libraries.
101 # uv.lock
102
103 # poetry
104 # Similar to Pipfile.lock, it is generally recommended to include poetry.lock in version control.
105 # This is especially recommended for binary packages to ensure reproducibility, and is more
106 # commonly ignored for libraries.
107 # https://python-poetry.org/docs/basic-usage/#commit-your-poetrylock-file-to-version-control
108 # poetry.lock
109 # poetry.toml
110
111 # pdm
112 # Similar to Pipfile.lock, it is generally recommended to include pdm.lock in version control.
113 # pdm recommends including project-wide configuration in pdm.toml, but excluding .pdm-python.
114 # https://pdm-project.org/en/latest/usage/project/#working-with-version-control
```

```
115 # pdm.lock
116 # pdm.toml
117 .pdm-python
118 .pdm-build/
119
120 # pixi
121 # Similar to Pipfile.lock, it is generally recommended to include pixi.lock in version control.
122 # pixi.lock
123 # Pixi creates a virtual environment in the .pixi directory, just like venv module creates one
124 # in the .venv directory. It is recommended not to include this directory in version control.
125 .pixi
126
127 # PEP 582; used by e.g. github.com/David-OConnor/pyflow and github.com/pdm-project/pdm
128 __pypackages__/_
129
130 # Celery stuff
131 celerybeat-schedule
132 celerybeat.pid
133
134 # Redis
135 *.rdb
136 *.aof
137 *.pid
138
139 # RabbitMQ
140 mnesia/
141 rabbitmq/
142 rabbitmq-data/
143
144 # ActiveMQ
145 activemq-data/
146
147 # SageMath parsed files
148 *.sage.py
149
150 # Environments
151 .env
152 .envrc
153 .venv
154 env/
155 venv/
156 ENV/
157 env.bak/
158 venv.bak/
159
160 # Spyder project settings
161 .spyderproject
162 .spyproject
163
164 # Rope project settings
165 .ropeproject
166
167 # mkdocs documentation
168 /site
169
170 # mypy
171 .mypy_cache/
172 .dmypy.json
173 dmypy.json
174
175 # Pyre type checker
176 .pyre/
177
178 # pytype static type analyzer
179 .pytype/
180
181 # Cython debug symbols
182 cython_debug/
183
184 # PyCharm
185 # JetBrains specific template is maintained in a separate JetBrains.gitignore that can
186 # be found at https://github.com/github/gitignore/blob/main/Global/JetBrains.gitignore
187 # and can be added to the global gitignore or merged into this file. For a more nuclear
188 # option (not recommended) you can uncomment the following to ignore the entire idea folder.
```

```
189 # .idea/
190
191 # Abstra
192 #   Abstra is an AI-powered process automation framework.
193 #   Ignore directories containing user credentials, local state, and settings.
194 #   Learn more at https://abstra.io/docs
195 .abstra/
196
197 # Visual Studio Code
198 #   Visual Studio Code specific template is maintained in a separate VisualStudioCode.gitignore
199 #   that can be found at https://github.com/github/gitignore/blob/main/Global/VisualStudioCode.gitignore
200 #   and can be added to the global gitignore or merged into this file. However, if you prefer,
201 #   you could uncomment the following to ignore the entire vscode folder
202 # .vscode/
203 # Temporary file for partial code execution
204 tempCodeRunnerFile.py
205
206 # Ruff stuff:
207 .ruff_cache/
208
209 # PyPI configuration file
210 .pypirc
211
212 # Marimo
213 marimo/_static/
214 marimo/_lsp/
215 __marimo__/
216
217 # Streamlit
218 .streamlit/secrets.toml
219
220 sessions/
221
222 output/
```